

Descifrando los Enigmas del Mercado Tecnológico: Un Análisis Exhaustivo de los Precios y Arquitecturas de Los Laptops

Andrés Felipe Obando Barriga

Colaborador académico

Colaboradora académica

Resumen—Durante el semestre, se realizó un estudio exhaustivo con el propósito de determinar la mejor computadora en términos de relación calidad-precio. Este análisis se basó en un conjunto de datos detallados que incluyeron variables esenciales como precio, marca, procesador, RAM, almacenamiento, tamaño de pantalla, sistema operativo y clasificación de los computadores. La metodología empleada abarcó la construcción de modelos estadísticos para comprender la relación entre el precio y las variables clave, así como el uso de técnicas de visualización de datos como histogramas y diagramas de dispersión para identificar patrones y tendencias. Además, se aplicó un enfoque multicriterio que determinó diversas características técnicas para facilitar la toma de decisiones informadas por parte de los consumidores.

Los resultados revelan cómo factores como la capacidad de RAM y el tipo de procesador influyen significativamente en el precio de las computadoras. Esta investigación proporciona una guía sólida para los consumidores al seleccionar un dispositivo que se ajuste a sus necesidades y presupuesto, optimizando así su inversión en tecnología. Al aplicar este enfoque, los consumidores pueden tomar decisiones más informadas y obtener el máximo valor de su compra tecnológica, lo que contribuye a una mejor satisfacción y rendimiento en sus actividades personales y profesionales.

Abstract—During the semester, an exhaustive study was conducted with the purpose of determining the best computer in terms of value for money. This analysis was based on a detailed data set that included essential variables such as price, brand, processor, RAM, storage, screen size, operating system, and computer classification. The methodology used included the construction of statistical models to understand the relationship between price and key variables, as well as the use of data visualization techniques such as histograms and scatter plots to identify

patterns and trends. In addition, a multi-criteria approach was applied that determined various technical characteristics to facilitate informed decision making by consumers.

The results reveal how factors such as RAM capacity and processor type significantly influence the price of computers. This research provides solid guidance for consumers in selecting a device that fits their needs and budget, thereby optimizing their technology investment. By applying this approach, consumers can make more informed decisions and get the most value from their technology purchase, contributing to better satisfaction and performance in their personal and professional activities.

Video Introducción:

I. PREGUNTA PROBLEMA

¿Cómo determinamos la mejor computadora en términos de calidad-precio, considerando variables clave y utilizando análisis estadísticos, visualización de datos y un enfoque multicriterio?

II. INTRODUCCIÓN

En la era digital actual, la elección de una computadora adecuada es fundamental para satisfacer las necesidades tanto personales como profesionales. Este estudio se basa en un conjunto de datos detallado que recopila información sobre diversas computadoras del mercado, incluyendo variables clave como el precio, la marca, el procesador, la RAM, el almacenamiento, el tamaño de pantalla y el sistema operativo. Mediante el uso de herramientas estadísticas y probabilísticas, se analizan estas características para comprender mejor las

relaciones entre ellas y su impacto en el rendimiento y el costo de los dispositivos.

Para visualizar y analizar los datos, se emplean técnicas como histogramas, diagramas de dispersión, diagramas de caja y gráficos de barras, que permiten identificar patrones y tendencias significativas. Estos análisis revelan cómo factores como la capacidad de RAM y el tipo de procesador influyen en el precio, y destacan las variaciones entre diferentes marcas y sistemas operativos. Finalmente, se emplea un enfoque multicriterio para determinar la mejor computadora en relación calidad-precio, ponderando diversas características técnicas junto con el precio. Este análisis permite identificar los modelos que ofrecen el equilibrio óptimo entre rendimiento y costo. Así, los consumidores pueden tomar decisiones informadas y seleccionar la computadora que mejor se adapta a sus necesidades y presupuesto, optimizando su inversión en tecnología.

La información de donde se obtuvo el DataSet, está en el foro.

III. OBJETIVOS

A. Objetivo general

Determinar la mejor computadora en relación calidad-precio mediante el análisis estadístico y probabilístico de un conjunto de datos que incluye variables clave como precio, marca, procesador, RAM, almacenamiento, tamaño de pantalla, sistema operativo y calificación de los computadores, utilizando técnicas de visualización de datos y un enfoque multicriterio.

B. Objetivo específicos:

1. Construir modelos estadísticos para analizar la relación entre el precio y variables permitiendo identificar patrones y tendencias.
2. Emplear técnicas de visualización de datos, como histogramas, diagramas de dispersión y gráficos de barras, para identificar patrones y relaciones significativas entre las diferentes características técnicas, el rating y el precio de las computadoras.
3. Utilizar un enfoque multicriterio que considera diversas características técnicas, facilitando así la toma de decisiones informadas para los consumidores.

IV. MARCO TEORICO.

- ¿Qué es un DataSet?

Un DataSet (conjunto de datos) es una colección organizada de datos, generalmente almacenada en una estructura tabular donde las columnas representan variables y las filas representan registros. Los DataSets son fundamentales en análisis de datos, ya que permiten almacenar, manipular y analizar grandes cantidades de información. Son utilizados en campos como la ciencia de datos, investigación científica y análisis de mercado.

Uso: Los DataSets se utilizan para entrenar modelos de aprendizaje automático, realizar análisis estadísticos, y generar informes y visualizaciones de datos.

Referencia: Han, J., Pei, J., & Kamber, M. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.

- Python y Teorías de Probabilidad

Python es un lenguaje de programación de alto nivel que se utiliza ampliamente en el análisis de datos y la estadística debido a sus bibliotecas potentes como NumPy, SciPy, y Pandas. Las teorías de probabilidad en Python permiten a los analistas modelar incertidumbres y realizar análisis estadísticos complejos, facilitando la implementación de modelos probabilísticos y simulaciones.

Uso: Python se utiliza para desarrollar algoritmos probabilísticos, realizar simulaciones estadísticas y analizar grandes conjuntos de datos.

Referencia: McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (2nd ed.). O'Reilly Media.

- Distribución de Poisson (Estadística)

La distribución de Poisson es una distribución de probabilidad discreta que describe la probabilidad de un número dado de eventos ocurridos en un intervalo fijo de tiempo o espacio, bajo la condición de que estos eventos ocurran con una tasa promedio constante e independientemente unos de otros. Es útil para modelar situaciones donde se cuentan eventos discretos, como llamadas telefónicas a un centro de llamadas o la aparición de defectos en una línea de producción.

Uso: Se utiliza en análisis de calidad, gestión de servicios, y estudios de eventos raros en diversas disciplinas.

Referencia: Haight, F. A. (1967). Handbook of the Poisson Distribution. John Wiley & Sons.

- Regresión Lineal

La regresión lineal es un método estadístico que modela la relación entre una variable dependiente y una o más variables independientes asumiendo que esta relación es lineal. Se utiliza para predecir valores y entender las relaciones entre variables. La fórmula básica es

$$Y=a+bX \text{ donde}$$

Y es la variable dependiente,

X es la variable independiente,

a es la intersección y

b es la pendiente.

Uso: Se utiliza en predicción de tendencias, análisis financiero, y modelado de relaciones entre variables en ciencias sociales y naturales.

Referencia: Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). Introduction to Linear Regression Analysis (5th ed.). Wiley.

- Regresión No Lineal

La regresión no lineal describe la relación entre variables dependientes e independientes mediante modelos no lineales. Estos modelos pueden capturar relaciones más complejas y variadas entre las variables, utilizando ecuaciones polinomiales, exponenciales, logarítmicas, entre otras.

Uso: Se utiliza cuando los datos muestran patrones que no pueden ser representados adecuadamente por una línea recta, como en estudios de crecimiento biológico, fenómenos físicos complejos y econometría avanzada.

Referencia: Seber, G. A. F., & Wild, C. J. (2003). Nonlinear Regression. Wiley-Interscience.

- ¿Qué es una Laptop?

Una laptop es un ordenador portátil que integra todos los componentes de un ordenador de escritorio en un solo dispositivo compacto y móvil. Incluye una pantalla, teclado, almacenamiento, memoria y unidad de procesamiento. Las laptops están diseñadas para la portabilidad y el uso en diferentes entornos, ofreciendo un equilibrio entre rendimiento y movilidad.

Uso: Se utilizan para trabajo, educación, entretenimiento y comunicación en cualquier lugar, facilitando la productividad y el acceso a la información.

Referencia: Barry, P. (2011). Laptops For Dummies. For Dummies.

- Arquitectura Computacional

La arquitectura computacional es el diseño conceptual y la estructura operacional de los sistemas de computación. Define cómo interactúan los componentes de hardware y software para procesar información. Incluye la estructura y el comportamiento de la CPU, la memoria y los sistemas de entrada/salida, así como las interconexiones entre ellos.

Uso: Es fundamental para diseñar sistemas de alto rendimiento, optimizar el funcionamiento de las computadoras y desarrollar nuevos dispositivos electrónicos.

Referencia: Patterson, D. A., & Hennessy, J. L. (2017). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann.

- ¿Qué es una RAM?

La memoria de acceso aleatorio (RAM) es un tipo de memoria volátil utilizada en computadoras para almacenar datos y código de programa que la CPU necesita acceder rápidamente mientras el dispositivo está en funcionamiento. La RAM permite que los sistemas informáticos ejecuten tareas simultáneamente y mejoren el rendimiento general al proporcionar acceso rápido a los datos.

Uso: Se utiliza para ejecutar programas y procesos en tiempo real, mejorar la velocidad de respuesta del sistema y soportar multitareas.

Referencia: Stallings, W. (2018). Computer Organization and Architecture (11th ed.). Pearson.

- ¿Qué es un Procesador?

El procesador, o unidad central de procesamiento (CPU), es el componente principal de una computadora que realiza operaciones aritméticas, lógicas y de control siguiendo las instrucciones de un programa. La CPU es esencial para la ejecución de tareas y el rendimiento general de un sistema computacional, ya que es responsable de la toma de decisiones y la ejecución de instrucciones.

Uso: Es utilizado para realizar cálculos, ejecutar programas y gestionar la comunicación entre los diferentes componentes de la computadora.

Referencia: Null, L., & Lobur, J. (2014). The Essentials of Computer Organization and Architecture (4th ed.). Jones & Bartlett Learning.

- Procesador Ryzen

Los procesadores Ryzen, desarrollados por AMD, son conocidos por su alto rendimiento y eficiencia energética. Utilizan la arquitectura Zen, que ha mejorado significativamente en términos de rendimiento y eficiencia comparado con las generaciones anteriores. Los Ryzen son populares tanto en sistemas de escritorio como en portátiles, ofreciendo capacidades avanzadas para tareas intensivas como juegos y creación de contenido.

Uso: Son ideales para usuarios que requieren un alto rendimiento en aplicaciones multitarea, juegos y procesamiento de gráficos intensivos.

Referencia: Mueller, S. (2020). Upgrading and Repairing PCs (22nd ed.). Que Publishing.

Procesador Intel

Intel es una de las principales compañías en la fabricación de procesadores. Sus CPUs son ampliamente utilizadas en ordenadores personales, servidores y dispositivos embebidos. Los procesadores Intel son conocidos por su rendimiento robusto, tecnologías innovadoras y fiabilidad. La arquitectura de los procesadores Intel ha evolucionado continuamente para ofrecer un mejor rendimiento y eficiencia energética.

Uso: Utilizados en una amplia gama de dispositivos, desde computadoras de consumo hasta servidores de alto rendimiento, proporcionando soluciones eficientes y escalables.

Referencia: Pabst, T., & Schmid, P. (2021). Inside Intel: The Architecture and Design of Intel CPUs. Springer.

- ¿Qué es un Histograma?

Un histograma es una representación gráfica de la distribución de un conjunto de datos. Muestra la frecuencia de datos dentro de intervalos específicos, permitiendo a los analistas ver la distribución y la variabilidad de los datos. Cada barra en un histograma representa la frecuencia de los datos en ese intervalo, proporcionando una visión clara de la distribución de los datos.

Uso: Se utiliza para visualizar la distribución de datos, identificar patrones y detectar valores atípicos en análisis estadísticos.

Referencia: Freedman, D., Pisani, R., & Purves, R. (2007). Statistics (4th ed.). W.W. Norton & Company.

- ¿Qué es un Diagrama de Caja y Bigotes?

Un diagrama de caja y bigotes, también conocido como boxplot, es una herramienta gráfica que muestra la distribución de un conjunto de datos basado en un resumen de cinco números: mínimo, primer cuartil, mediana, tercer cuartil y máximo. Este diagrama ayuda a identificar la variabilidad, la simetría, los valores atípicos y la dispersión de los datos.

Uso: Se utiliza para comparar distribuciones entre diferentes grupos y detectar valores atípicos en los datos.

Referencia: McGill, R., Tukey, J. W., & Larsen, W. A. (1978). Variations of Box Plots. The American Statistician, 32(1), 12-16.

- ¿Qué es un Diagrama de Torta?

Un diagrama de torta, o gráfico circular, es una representación gráfica que muestra las proporciones de diferentes categorías de datos como segmentos de un círculo. Cada segmento representa una categoría y su tamaño es proporcional a la frecuencia o cantidad de esa categoría en el conjunto de datos. Es útil para visualizar la composición y comparación relativa de las partes de un todo.

Uso: Se utiliza para mostrar proporciones y comparaciones relativas de diferentes categorías en un conjunto de datos.

Referencia: Knaflitz, C. N. (2015). Storytelling with Data: A Data Visualization Guide for Business Professionals. Wiley.

- ¿Qué es un Diagrama de Dispersión?

Un diagrama de dispersión es una representación gráfica que muestra la relación entre dos variables. Cada punto en el gráfico representa un par de valores de las variables, permitiendo visualizar patrones, tendencias y correlaciones. Es una herramienta fundamental en análisis de regresión y estadística descriptiva.

Uso: Se utiliza para identificar relaciones y correlaciones entre variables, y para explorar la fuerza y la dirección de las relaciones.

Referencia: Chambers, J. M., Cleveland, W. S., Kleiner, B., & Tukey, P. A. (1983). Graphical Methods for Data Analysis. Duxbury Press.

- Asus

Asus es una compañía taiwanesa de tecnología conocida por fabricar una amplia gama de productos electrónicos, incluyendo computadoras portátiles, componentes de PC, teléfonos inteligentes y más. Asus es reconocida por su innovación, calidad y rendimiento, siendo una marca popular entre los entusiastas de la tecnología y los gamers.

Uso: Proporciona productos tecnológicos de alta calidad para consumidores, profesionales y entusiastas del gaming, ofreciendo dispositivos fiables y avanzados.

Referencia: Kim, L., & Mauborgne, R. (2005). Blue Ocean Strategy: How to Create Uncontested Market Space and Make the Competition Irrelevant. Harvard Business Review Press.

- Acer.

Acer es una empresa multinacional taiwanesa que fabrica productos de tecnología, incluyendo computadoras portátiles, de escritorio, tabletas, monitores y otros dispositivos. Su enfoque principal está en ofrecer productos de alta calidad a precios competitivos. En el contexto de las computadoras personales, Acer es conocido por ofrecer una amplia gama de productos para diferentes segmentos de mercado, desde computadoras portátiles básicas hasta modelos de alta gama para jugadores y profesionales creativos (Acer Inc., 2024).

- Lenovo.

Es una empresa tecnológica multinacional con sede en China. Es una de las mayores fabricantes de computadoras personales del mundo. Ofrece una variedad de productos, que incluyen computadoras portátiles, de escritorio, tabletas, smartphones y servidores. En el ámbito de las computadoras personales, Lenovo es conocido por su enfoque en la innovación y la calidad de sus productos. Ofrece opciones para consumidores individuales, empresas y sectores educativos (Lenovo Group Limited, 2024).

- HP (Hewlett-Packard)

Es una empresa estadounidense que fabrica una amplia gama de productos de tecnología, incluyendo computadoras personales, impresoras, servidores, dispositivos de almacenamiento, software y servicios relacionados. HP es reconocida por su larga historia en la industria de la tecnología y su compromiso con la innovación. En el contexto de las computadoras personales, HP ofrece una variedad de productos para diferentes necesidades y presupuestos, desde computadoras portátiles y de escritorio hasta estaciones de

trabajo de alto rendimiento (HP Development Company, L.P., 2024).

V. METODOLOGÍA

El objetivo de este informe es realizar un análisis estadístico exhaustivo utilizando Python para comprender la relación entre las características de las laptops, sus precios y las calificaciones de los usuarios. El estudio comienza con la carga del dataset, el cual contiene información detallada sobre diversas especificaciones técnicas de las laptops, sus precios y las calificaciones otorgadas por los usuarios. La primera etapa del análisis implica una exploración inicial de los datos, utilizando pandas para cargar el dataset y funciones como `'head()'`, `'info()'`, y `'describe()'` para obtener una visión general de su estructura y contenido. Es fundamental identificar y manejar valores nulos y duplicados en esta fase para asegurar la calidad de los datos, aplicando técnicas de limpieza como la eliminación de filas con valores nulos utilizando `'dropna()'`. Además, se realizan conversiones de tipos de datos cuando es necesario para facilitar el análisis posterior.

Una vez que los datos están limpios, se procede con un análisis descriptivo utilizando visualizaciones. Se crean histogramas para observar la distribución de precios y calificaciones de las laptops, empleando matplotlib para graficar los datos y facilitar la interpretación visual. Por ejemplo, un histograma de los precios ayuda a identificar la variabilidad y tendencias en los costos de las laptops. Simultáneamente, se generan diagramas de torta para visualizar la proporción de diferentes marcas de laptops en el dataset, lo cual proporciona una perspectiva clara sobre la diversidad de marcas representadas. Estos gráficos permiten identificar patrones iniciales y áreas de interés que merecen un análisis más profundo.

El siguiente paso es realizar un análisis de correlación para entender las relaciones entre las variables numéricas del dataset. Se calcula una matriz de correlación y se visualiza utilizando un mapa de calor (heatmap) con seaborn, donde las correlaciones fuertes (positivas o negativas) se destacan claramente. Este análisis ayuda a identificar cuáles características técnicas de las laptops tienen mayor influencia sobre el precio y las calificaciones, proporcionando una base para los modelos de regresión.

Para el análisis predictivo, se construyen modelos de regresión lineal y no lineal. En el caso de la regresión lineal, se seleccionan características técnicas como variables

independientes y se define el precio o la calificación como la variable dependiente. Utilizando sklearn, se divide el dataset en conjuntos de entrenamiento y prueba con `'train_test_split'`, y se ajusta un modelo de regresión lineal con `'LinearRegression()'`. La precisión del modelo se evalúa mediante métricas como el coeficiente de determinación (R^2) y el error cuadrático medio (MSE). Para capturar relaciones más complejas que no se ajustan bien a un modelo lineal, se exploran modelos de regresión no lineal, como la regresión polinómica, utilizando la clase `'PolynomialFeatures'` de sklearn para transformar las características antes de ajustar el modelo.

Además del análisis de regresión, se incorporan teorías de probabilidad para entender la distribución y variabilidad de los datos. Por ejemplo, se puede utilizar la distribución normal para modelar la variabilidad de los precios de las laptops y calcular probabilidades asociadas a rangos de precios específicos. Este análisis probabilístico se complementa con histogramas y gráficos de densidad para visualizar cómo se distribuyen los precios y las calificaciones.

Finalmente, se interpretan los resultados obtenidos de los modelos de regresión y el análisis descriptivo. Se discuten las implicaciones prácticas de los hallazgos, como cuáles características son más valoradas por los usuarios y cómo afectan el precio de las laptops. También se proponen recomendaciones basadas en los resultados del análisis para fabricantes de laptops y consumidores. El informe concluye con una reflexión sobre las limitaciones del estudio, como la calidad del dataset y la necesidad de datos adicionales para mejorar la precisión de los modelos predictivos. En resumen, este informe utiliza un enfoque estructurado y metodológico para proporcionar una comprensión profunda de los factores que influyen en los precios y calificaciones de las laptops, utilizando técnicas de análisis de datos y visualización en Python.

VI. DESARROLLO

Corte 1:

Para el primer corte del estudio, se seleccionó un conjunto de datos que contiene una amplia variedad de variables para analizar. Esto significa que el conjunto de datos incluiría información diversa sobre diferentes aspectos de las computadoras, como el precio, la marca, el procesador, la cantidad de RAM, el almacenamiento, el tamaño de la pantalla, el sistema operativo y posiblemente otros detalles relevantes.

La elección de un conjunto de datos con esta variedad de variables permitirá realizar un análisis integral y detallado de las computadoras disponibles en el mercado.

Con esta variedad de datos, será posible realizar un análisis completo de las relaciones entre las diferentes variables y su impacto en el rendimiento y el costo de las computadoras. Además, permitirá emplear una amplia gama de técnicas estadísticas y probabilísticas para comprender mejor estos patrones y tendencias.

Programación:

```
from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[ ] datos = pd.read_csv('/content/drive/MyDrive/Datos/laptop_price.csv', encoding='latin-1')
[ ] datos.head(1002)

[ ] datos.describe()
```

Figura 1: Programación 1.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimientos:

En Google Colab, una plataforma de desarrollo en la nube basada en Python, la importación de la biblioteca "drive" y el montaje de Google Drive mediante `drive.mount('/content/drive')` facilitan el acceso a archivos almacenados en la nube desde el entorno de Colab. Este proceso permite que los usuarios trabajen con conjuntos de datos alojados en Google Drive de manera eficiente. Una vez montado, se asigna una variable, denominada DataFrame en el contexto de la biblioteca pandas, para representar los datos. Este DataFrame permite la manipulación y análisis de los datos de manera efectiva.

Posteriormente, al utilizar `data.head(1002)`, se muestran las primeras 1002 filas del DataFrame "data", proporcionando una vista previa de los datos y permitiendo comprender su estructura y contenido. Finalmente, la función `data.describe()` genera estadísticas descriptivas sobre los datos almacenados en el DataFrame, como recuentos, medios, desviaciones estándar y percentiles. Estas estadísticas son fundamentales para comprender la distribución y las características de los datos, lo que facilita el proceso de análisis y la toma de decisiones basadas en datos dentro de Google Colab.

Ejecución:

	laptop_ID	Inches	Price_euros
count	1303.000000	1303.000000	1303.000000
mean	660.155794	15.017191	1123.686992
std	381.172104	1.426304	699.009043
min	1.000000	10.100000	174.000000
25%	331.500000	14.000000	599.000000
50%	659.000000	15.600000	977.000000
75%	990.500000	15.600000	1487.880000
max	1320.000000	18.400000	6099.000000

Figura 2: Ejecución 1.

Fuente: Propia, echo en Google Colab.

[xK](#)

Conclusiones:

Al analizar el tamaño de la pantalla de las computadoras en el conjunto de datos, observamos que el promedio es de aproximadamente 15.02 pulgadas. Este valor indica que la mayoría de las computadoras tienen pantallas que rondan este tamaño, con la mayoría de ellas ubicadas entre las 14 y 16 pulgadas. Sin embargo, existe una variabilidad en el tamaño de las pantallas, con la más pequeña registrada en 10,1 pulgadas y la más grande en 18,4 pulgadas. Este análisis proporciona una idea de la diversidad de opciones disponibles en cuanto al tamaño de la pantalla de las computadoras.

En cuanto a los precios, que están expresados en euros, observamos que varían significativamente en el conjunto de datos. El rango de precios oscila entre 599 y 1487 euros, con un valor mínimo de 174 euros y un máximo de 6099 euros. Al considerar los percentiles, como el 25%, 50% y 75%, podemos identificar la distribución de los precios. Por ejemplo, el percentil 50%, que también es conocido como la mediana, se encuentra en 699 euros, lo que indica que la mitad de las computadoras tienen precios por encima de este valor y la otra mitad por debajo. La desviación estándar de 699 euros sugiere una dispersión de los precios alrededor de la media, lo que significa que algunos modelos tienen precios significativamente más altos o bajos que la media. Esta información se puede visualizar mediante un diagrama de caja (boxplot), que muestra los cuartiles, los valores atípicos y la distribución de los precios de manera más clara y concisa.

Programación:

Tipos de datos en el DataSet:

```
[ ] datos.dtypes
```

Figura 3: Programación 2.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimientos:

Este código realiza una inspección detallada de los tipos de datos presentes en cada columna del conjunto de datos, que se llama "datos". Utilizando la función `dtypes`, se obtiene información sobre el tipo de variable presente en cada columna, lo que incluye datos como el nombre del producto, la compañía fabricante y otros atributos relevantes de las laptops. Esta inspección es crucial para comprender la naturaleza de los datos con los que se está trabajando, lo que permite realizar análisis y manipulaciones específicas de acuerdo con los tipos de datos presentes en el conjunto.

Ejecución:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 13 columns):
# Column Non-Null Count Dtype
---
0 laptop_ID 1303 non-null int64
1 Company 1303 non-null object
2 Product 1303 non-null object
3 TypeName 1303 non-null object
4 Inches 1303 non-null float64
5 ScreenResolution 1303 non-null object
6 Cpu 1303 non-null object
7 Ram 1303 non-null object
8 Memory 1303 non-null object
9 Gpu 1303 non-null object
10 OpSys 1303 non-null object
11 Weight 1303 non-null object
12 Price_euros 1303 non-null float64
dtypes: float64(2), int64(1), object(10)
memory usage: 132.5+ KB
```

Figura 4: Ejecución 3.

Fuente: Propio, echo en Google Colab.

[xK](#)

Conclusiones:

Claro, en el conjunto de datos, algunas columnas contienen información de tipo "object", lo que indica que pueden contener cadenas de texto u otros tipos de datos no numéricos. Sin embargo, dado que esta información no se utilizará en el análisis posterior, se ha decidido dejar estas columnas en su formato original de cadena de texto. Por otro lado, las columnas que contienen información crítica para el análisis, como "laptop_ID", "Pulgadas", "Price_euros" y "Company", se han asignado a sus respectivos tipos de datos numéricos o de cadena de texto según corresponda. Estas columnas son fundamentales para el análisis y la toma de decisiones posteriores, por lo que es importante que estén en el formato adecuado para su manipulación y análisis.

Programación:

```
important_features=["laptop_ID","Company","Product","TypeName","Inches","ScreenResolution","Cpu","Ram","Memory","Gpu","OpSys","Weight","Price_euros"]
datos_data=datos.loc[:,important_features]
datos_data.describe()
```

Figura 5: Programación 3.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimiento:

El código comienza importando los nombres de las columnas que son consideradas como características importantes para el análisis, las cuales incluyen "Laptop_ID", "Company", "Product", "TypeName", "Pulgadas", "ScreenResolution", "Cpu", "Ram", "Memory", "Gpu", "OpSys", "Peso" y "Precio_euros". Luego, utilizando el método "loc[]", se seleccionan todas las filas del conjunto de datos original y solo las columnas especificadas en la lista "important_features", lo que crea un nuevo DataFrame llamado "datos_data". Finalmente, se utiliza el método "describe()" para obtener estadísticas descriptivas sobre las variables numéricas en el nuevo DataFrame y se muestra una vista previa de las primeras 1002 filas de este DataFrame para visualizar los datos y comprender mejor su estructura y contenido. Este proceso es esencial para preparar los datos para el análisis posterior y facilitar la comprensión de los patrones y tendencias presentes en el conjunto de datos.

Ejecución:

	laptop_ID	Inches	Price_euros
count	1303.000000	1303.000000	1303.000000
mean	660.155794	15.017191	1123.686992
std	381.172104	1.426304	699.009043
min	1.000000	10.100000	174.000000
25%	331.500000	14.000000	599.000000
50%	659.000000	15.600000	977.000000
75%	990.500000	15.600000	1487.880000
max	1320.000000	18.400000	6099.000000

Figura 6: Ejecución 3.

Fuente: Propia, echo en Google Colab.

[xK](#)

Conclusiones:

Al ejecutar este código, se observa que los datos se han organizado y se han seleccionado las características importantes para el análisis, lo que nos permite comenzar a trabajar con un conjunto de datos más específico y relevante. Al limitar el número de filas a las primeras 1002 del DataFrame, se proporciona una vista más manejable y representativa de los datos, lo que facilita el análisis y la identificación de patrones. Esto nos permite comprender mejor la estructura y la distribución de nuestros datos, así como también nos da una idea inicial de las relaciones entre las diferentes variables. En resumen, este proceso de organización y selección de datos sienta una base sólida para el análisis posterior, permitiéndonos explorar y entender mejor las características y tendencias presentes en nuestro conjunto de datos de portátiles.

Programación:

```
datos_data.head(1002)
```

Figura 7: Programación 4.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimiento:

Utilizando el DataFrame "datos_data", que contiene las primeras 1002 filas de nuestro conjunto de datos ordenados y

con las características importantes seleccionadas, podemos acceder a los datos específicos de cada fila y columna. Al tomar las primeras 1002 filas, aseguramos que estamos trabajando con una muestra representativa de nuestro conjunto de datos.

Ejecución:

laptop_ID	Company	Product	Type/Name	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OsType	Weight	Price_euros	
0	1	Apple	MacBook Pro	Ultrabook	13.3	P5 Panel Retina Display 2560x1600	Intel Core i5 1.3GHz	8GB	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37kg	1539.00
1	2	Apple	Macbook Air	Ultrabook	13.3	1440x900	Intel Core i5 1.6GHz	8GB	128GB Flash Storage	Intel HD Graphics 600	macOS	1.34kg	898.94
2	3	HP	250 G6	Notebook	15.6	Full HD 1520x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD	Intel HD Graphics 620	No OS	1.84kg	575.00
3	4	Apple	Macbook Pro	Ultrabook	15.4	P5 Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16GB	512GB SSD	AMD Radeon Pro 455	macOS	1.83kg	2537.45
4	5	Apple	Macbook Pro	Ultrabook	13.3	P5 Panel Retina Display 2560x1600	Intel Core i5 1.3GHz	8GB	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37kg	1085.60
...	...	...	...	...	...	...	...	...	...	...	...	...	
997	1001	Acer	Aspire E5-513	Notebook	15.6	1366x768	Intel Pentium Quad Core M4200 1.1GHz	4GB	500GB HDD	Intel HD Graphics 520	Windows 10	2.4kg	380.00
998	1002	Asus	Rog GL503GE-S074	Gaming	15.6	Full HD 1920x1080	Intel Core i7 7700HQ 2.8GHz	16GB	256GB SSD + 1TB HDD	Nvidia GeForce GTX 1050 Ti	Windows 10	2.5kg	1799.00
999	1003	HP	EliteBook 840	Ultrabook	14.0	Full HD 1920x1080	Intel Core i7 7500U 2.7GHz	8GB	512GB SSD	Intel HD Graphics 620	Windows 10	1.48kg	2089.00
1000	1004	Acer	Nitro 5	Gaming	15.6	P5 Panel Full HD 1920x1080	Intel Core i7 7700HQ 2.8GHz	8GB	128GB SSD + 1TB HDD	Nvidia GeForce GTX 1050	Windows 10	2.7kg	1260.00
1001	1005	HP	ENVY+	Notebook	13.3	P5 Panel Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD	Intel HD Graphics 620	Windows 10	1.34kg	1089.00

1002 rows x 13 columns

Figura 8: Ejecución 4.

Fuente: Propia, echo en Google Colab.

[xK](#)

Programación:

```
plot = datos_data["Inches"].hist(edgecolor='black', linewidth=1.2)
plot.set_title('Tamaño en pulgadas')
plt.show()
```

Figura 9: Programación 5.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimientos:

Para crear un histograma basado en la columna "Pulgadas" de nuestro DataFrame, primero asignamos los datos de esta columna a una variable, como "pulgadas_data". Luego, empleamos métodos de personalización de la biblioteca de visualización de datos que estamos utilizando, como matplotlib o seaborn, para ajustar la apariencia del histograma. Esto implica establecer el título del histograma, modificar el color del borde y el color de las barras según nuestras preferencias de diseño. Una vez configurado, mostramos el histograma en la interfaz de usuario para su visualización, permitiéndonos así examinar la distribución de los tamaños de pantalla de las computadoras de manera más clara y comprensible.

Ejecución:

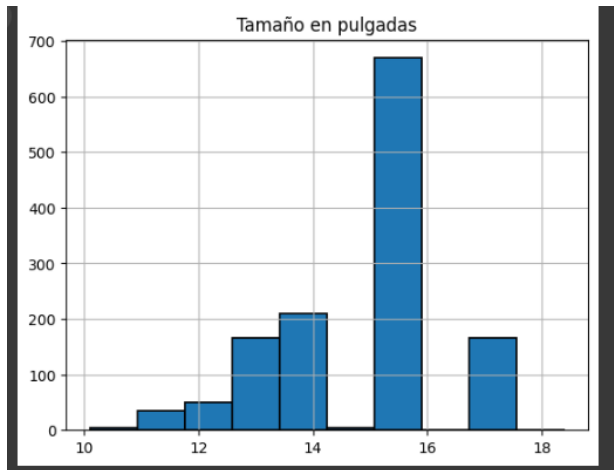


Figura 10: Ejecución 5.

Fuente: Propia, echo en Google Colab.

[xK](#)

Conclusiones:

Esta interpretación de la gráfica proporciona una comprensión detallada de la relación entre el tamaño de la pantalla y el rango de precios de las computadoras portátiles. En el eje x, que representa los diferentes tamaños de pantalla, se observa que los tamaños más comunes son 10, 12, 14, 16 y 18 pulgadas. En el eje y, que muestra el rango de precios, se destaca que el intervalo de precios más popular se encuentra entre 600 y 800 euros. Además, se indica que la mayoría de los visitantes, alrededor del 40%, prefieren las portátiles de 15 pulgadas, lo que sugiere que estas son las más demandadas. Se infiere que las laptops de 17 pulgadas y 13 pulgadas también son comunes, ya que son mencionados como las siguientes más probables en términos de popularidad. Además, se destaca que las laptops de precios bajos son más comunes que las de precios altos, lo que sugiere que los consumidores tienden a preferir laptops mayores. Por último, se resalta la relación entre el tamaño de la pantalla y el precio, observando que a medida que aumenta el tamaño de la pantalla, también lo hace el precio, lo que indica una compensación positiva entre estas dos variables.

Programación:

```
fig = plt.figure(figsize =(10, 7))

# Creating plot
plt.boxplot(datos_data['Price_euros'])
plt.title("Boxplot Precio en Euros [€]")
plt.grid()
#plt.xlabel('X (m)')
plt.ylabel('€')
# show plot
plt.show()
```

Figura 10: Programación 6.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimiento:

Se utiliza la biblioteca de “matplotlib” esta es usada para hacer un diagrama de caja en donde vamos a tomar los datos de “Price_euros”. Después de esto se crea la figura por medio de la biblioteca con un tamaño de 10 y 7 pulgadas, este es el espacio en donde se va a crear el diagrama. Después usamos “datos_data” con los precios de la tabla para si obtener la columna ya deseada y hacer el diagrama de caja mostrando así la distribución de los precios, ya con esto se crea el título con una cuadrícula para el grafico.

Ejecución:

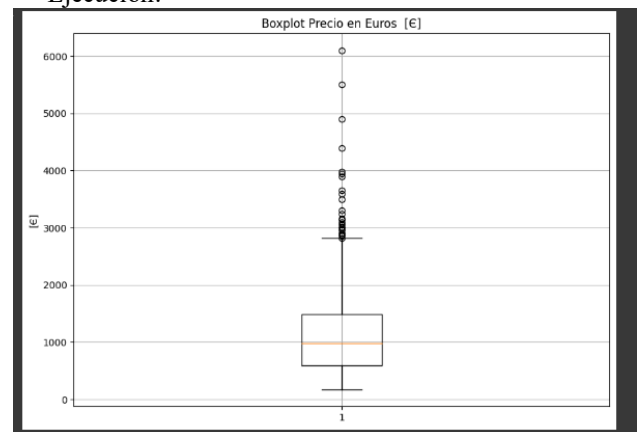


Figura 11: Ejecución 6.

Fuente: Propia, echo en Google Colab.

[xK](#)

Conclusiones:

Al analizar el boxplot de los precios en euros de las portátiles, observamos que la media se sitúa alrededor de los 1000 euros. Sin embargo, es importante destacar que los valores más comunes están por encima del tercer cuartil, en el rango de

2700 a 3000 euros, indicando que estas son las inversiones más frecuentes para la compra de portátiles. Los portátiles en el rango de 3000 a 4000 euros son menos comunes, y es raro encontrar compras que superen este rango de precios. El precio más alto registrado se aproxima a los 6000 euros. Esta distribución de precios sugiere que las marcas y sus características pueden influir significativamente en los precios, con ciertas marcas premium o modelos de alta gama alcanzando precios más elevados.

Programación:

```
import numpy as np

important_features = ['laptop_ID', 'Company', 'Product', 'TypeName', 'Inches', 'ScreenResolution']
datos_data = datos.loc[:, important_features]
print("Estadísticos sobre la variable 'Price_euros'")

def medidas_posicion(columna):
    media = np.mean(columna)
    mediana = np.median(columna)
    percentil_25 = np.percentile(columna, 25)
    percentil_75 = np.percentile(columna, 75)

    return media, mediana, percentil_25, percentil_75

def diagrama_tallos_hojas(columna):
    tallos = [int(x // 10) for x in columna]
    hojas = [int(x % 10) for x in columna]
```

Figura 12: Programación 7.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimientos:

Importamos nuevamente la misma librería “import numpy as np”, nombramos nuevamente nuestra lista “important features” e imprimimos “Estadísticos sobre la variable 'Price_euros'”. Terminando esta parte usamos “medidas_posicion(columna)” donde nos ayudara a encontrar la media, mediana y el cuartil superior e inferior para cada columna, estos resultados los pondremos en variables. Ahora definimos una función llamada “diagrama_tallos_hojas” esto creara un diagrama de tallos y hojas para la distribución de cada dato en una columna dada. En “tallos = [int(x // 10) for x in columna]” estamos creando los tallos van a ser cada valor de la columna entre 10 siendo esta una división entera por eso los dobles // tomando la parte entera del resultado, proporcionando así los tallos del diagrama. Para las hojas se ejecuta “hojas = [int(x % 10) for x in columna]” lo que calculara el residuo de la división entre 10 por cada valor en la columna dando así las hojas del diagrama. Teniendo ya los tallos y las hojas creamos la variable “diagrama” que almacenara estos datos en una lista, el bucle que estamos creando por medio de los tallos (t) y las hojas (h) y la función “zip” asignaremos las hojas a los tallos correspondientes en el diccionario, en donde si el tallo no está

presente el diccionario creara una nueva entrada y consigo una lista vacía para las hojas, para finalizar esta parte se imprimirá el diagrama ordenado por los tallos, mostrando los tallos seguidos de las hojas que corresponda. Para la última parte del código creamos un array de numpy con los datos de los precios, como también llamaremos la media, mediana y los dos cuartiles para ser calculados y finalmente imprimimos el diagrama de tallos y hojas.

Ejecución:

```
Estadísticos sobre la variable "Price_euros"
Media: 1123.6869915579432
Mediana: 977.0
Percentil 25: 599.0
Percentil 75: 1487.88
```

```
Diagrama de Tallos y Hojas:
17 | 4
19 | 1 9 9 6
20 | 2 9 9 9 9
21 | 0
22 | 9 4 9 9 9
23 | 9
24 | 4 9 8 5 9 9 9
25 | 8 9 2 5
26 | 0 9 5 9 5
27 | 7 5 4 4 9 4 0 8 2
28 | 6 8 7 9 9 9
29 | 2 9 9 5 8 8 8 9 9 7 7 9
30 | 9 9 6 4 4 9 9
31 | 9 8
32 | 1 9 9 5 4
33 | 0 0 9 3 9
34 | 4 9 4 8 9 7 9 0 9 9 4 9 9 8 9 5
35 | 9 9 5 9
36 | 7 9 9 3 9 4 9 9 8 1 9 9 9
37 | 5 9 5 9 9 9 9
38 | 9 4 9 9 9 9 5 9 0 7
39 | 3 8 9 5 8 0 8 7 9 9 8 3 8 8 9
40 | 0 3 3 9 9
41 | 8 5 4 9 2 9 0
42 | 8 6 9 9
43 | 9 9 8 9 9 5 5
44 | 9 7 7 9 5 4 9 9 1 3 9 9 3
45 | 1 9 9 9 9 0 0 5 9 9 9 9
```

Figura 13: Ejecución 7.

Fuente: Propia, echo en Google Colab.

[xK](#)

Conclusiones:

La mediana de los precios de las portátiles es de 977 euros, lo que sugiere que la mitad de las portátiles tienen un precio igual o inferior a este valor. Sin embargo, la media es de

1123,69 euros, lo que indica que existen algunos valores atípicos elevados que están influyendo en la distribución de los precios. El percentil 25, que es de 599 euros, indica que el 25% de las portátiles cuestan 599 euros o menos. Por otro lado, el percentil 75, que es de 1487,88 euros, muestra que el 25% de las portátiles tienen precios superiores a este valor, lo que señala una considerable variabilidad en los precios.

En el diagrama de tallo y hojas, se observa una alta concentración de precios bajos en la parte izquierda, donde los tallos tienen el mayor número de hojas. Esto significa que hay una cantidad significativa de portátiles en los rangos de precios más bajos. A medida que nos desplazamos hacia la parte inferior del diagrama, los tallos tienen menos hojas, indicando una menor frecuencia de laptops en los rangos de precios más altos. Este patrón muestra que la mayoría de las computadoras portátiles están agrupadas en los rangos de precios más accesibles, mientras que los precios más altos son menos comunes. El diagrama de tallo y hojas proporciona una visualización detallada de la distribución de los precios, destacando las áreas de mayor concentración.

Programación:

```
import numpy as np

important_features = ['laptop_ID', 'Company', 'Product', 'TypeName', 'Inches', 'ScreenResolution', 'Cpu', 'Ram', 'Memory', 'Gpu', 'O']
datos_data = datos.loc[:, important_features]

def medidas_posicion(columna):
    media = np.mean(columna)
    mediana = np.median(columna)
    percentil_25 = np.percentile(columna, 25)
    percentil_75 = np.percentile(columna, 75)

    return media, mediana, percentil_25, percentil_75

def diagrama_tallos_hojas(columna):
    tallos = [int(x // 10) for x in columna]
    hojas = [int(x % 10) for x in columna]

    diagrama = {}
    for t, h in zip(tallos, hojas):
        if t not in diagrama:
            diagrama[t] = []
        diagrama[t].append(h)

    for t, h in sorted(diagrama.items()):
        print(f'{t} | {" ".join(map(str, h))}')

for t, h in sorted(diagrama.items()):
    print(f'{t} | {" ".join(map(str, h))}')

datos1 = np.array(datos_data['Inches'])
media, mediana, p25, p75 = medidas_posicion(datos_data['Inches'])

print(f'Media: {media}')
print(f'Mediana: {mediana}')
print(f'Percentil 25: {p25}')
print(f'Percentil 75: {p75}')

print("\nDiagrama de Tallos y Hojas para Inches:")
diagrama_tallos_hojas(datos1)
```

Figura 14: Programación 8.

Fuente: Propia, echo en Google Colab.

[xk](#)

Procedimiento:

Importamos nuevamente la biblioteca "numpy" como "np" para realizar cálculos numéricos. A continuación, seleccionamos las categorías importantes de nuestro DataFrame original, almacenadas en "important_features", y utilizamos el método ".loc[]" para seleccionar las filas y columnas especificadas, creando así un nuevo DataFrame llamado "datos_data". Posteriormente, definimos "medidas_posicion" para nuestras columnas clave, como "Pulgadas", utilizando la biblioteca "np" para calcular estadísticas como las medidas media, la mediana y los percentiles 25 y 75. Usamos un bucle "for" para iterar sobre estas medidas y obtener nuestro diagrama, específicamente un boxplot, que se diferencia del código anterior en que ahora nos enfocamos en la columna de las pulgadas ("Pulgadas") para analizar la distribución y variabilidad del tamaño de las pantallas de las laptops. Este enfoque nos permite visualizar y entender mejor la distribución de los tamaños de pantalla en el conjunto de datos.

Ejecución:

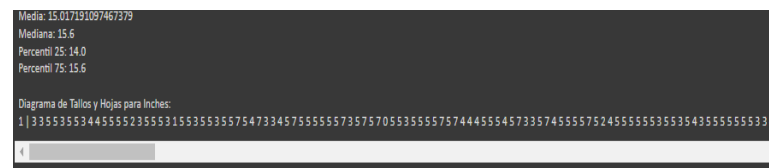


Figura 15: Ejecución 8.

Fuente: Propia, echo en Google Colab.

[xk](#)

Conclusiones:

Media: la media de las pulgadas de los laptops ronda entre los 15.01 lo cual es lógico teniendo en cuenta nuestro grafico anterior donde se ve que la barra superior es de 15. Mediana: la mediana indica que es de 15.02 esto indica que el 50% de los datos tiene un valor entre 15.06 pulgadas o menos también se conoce la media como el percentil de 50. Percentil de 25: es de 14,0 pulgadas indica que un cuarto de los valores está por debajo o igual a esta medida, en la gráfica se puede ver que la segunda barra más grande es esta lo cual nos indica que los datos que estamos sacando están siendo verdaderos. Percentil de 75: es de 15.6 pulgadas indicando así que el 75% de los datos tienen un mismo valor o menos. El percentil de 75 y la mediana es la misma esto indica que hay una concentración de datos debajo de la media, esto quiere decir que hay una asimetría en la distribución de los datos hacia valores más bajos. El

diagrama de hojas su único tallo es de 1 ya que las medidas no sobrepasan de esta unidad ya que están en pulgadas.

Programación:

```
df = datos_data['Company'].value_counts()
print(df)
```

Figura 16: Programación 9.

Fuente: Propia, echo en Google Colab.

[xK](#)

Procedimientos:

En el DataFrame llamado `datos\_data`, seleccionamos la columna "Empresa". Esta columna contiene una lista de nombres de compañías. Para contar cuántas veces aparece cada compañía en esta columna, utilizamos el método `.value\_counts()`. Este método analiza la columna "Company" y devuelve una Serie de pandas, donde cada índice es un nombre de compañía y cada valor es el número de veces que dicha compañía aparece en la columna. Esta Serie resultante se guarda en una nueva variable llamada `df`. Finalmente, imprimimos el contenido de `df` para visualizar los nombres de las compañías junto con su frecuencia de aparición.

Ejecución:

```
Dell      297
Lenovo    297
HP        274
Asus      158
Acer      103
MSI       54
Toshiba   48
Apple     21
Samsung   9
Razer     7
Mediacom  7
Microsoft 6
Xiaomi    4
Vero      4
Chuwi     3
Google    3
Fujitsu   3
LG        3
Huawei     2
Name: Company, dtype: int64
```

Figura 17: Ejecución 9.

Fuente: Propia, echo en Google Colab.

[xK](#)

Conclusiones:

En el DataFrame llamado `datos\_data`, seleccionamos la columna "Empresa". Esta columna contiene una lista de nombres de compañías. Para contar cuántas veces aparece cada compañía en esta columna, utilizamos el método `.value\_counts()`. Este método analiza la columna "Company" y devuelve una Serie de pandas, donde cada índice es un nombre de compañía y cada valor es el número de veces que dicha compañía aparece en la columna. Esta Serie resultante se guarda en una nueva variable llamada `df`. Finalmente, imprimimos el contenido de `df` para visualizar los nombres de las compañías junto con su frecuencia de aparición.

Programación:

```
data=datos_data['Company']
n=data.value_counts(sort=False)
N=n.cumsum()
f=100*data.value_counts(sort=False)/len(data)
F=100*f.cumsum()/f.sum()
```

Figura 18: Programación 10.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Llamamos a nuestro "datos_data" con la columna de "Company" ahora teniendo nuestros datos, usaremos la siguiente línea "n=data.value_counts(sort=False)" esta línea cuenta la frecuencia de cada una de los datos de "Company" y los almacena en la serie llamada "n" con un parámetro "sort=False" evita que los valores se ordenen por frecuencia. Ahora "N=n.cumsum()" acá calcula las frecuencias acumuladas. Con la siguiente línea de código "f=100*data.value_counts(sort=False)/len(data)" calculara la frecuencia relativa de cada único valor de la serie "data" donde esta se calcula dividiendo la frecuencia de cada valor por el número total de elementos de "data" y se multiplican por 100 de esta forma se obtiene un porcentaje. Estos datos se guardarán en f. Y en la última parte de este código "F=100*f.cumsum()/f.sum()" que es similar a la línea anterior la diferencia es que se van a calcular las frecuencias relativas acumuladas se utilizara primero "cumsum()" que hará la sumara acumulativa de las frecuencias de nuestra variable f, luego estas se divide por la suma total de frecuencias relativas y se

multiplica por 100 para obtener un porcentaje acumulado. Esto se guardará en F.

Programación:

```
#Crear el diccionario y el dataframe de la tabla de frecuencias
tablafrec={'n':n,'N':N,'f(%)':f,'F(%)':F}
df1 = pd.DataFrame(tablafrec,columns=['n','N','f(%)','F(%)'])
print(df1)
```

Figura 19: Programación 11.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Vamos a crear una tabla de frecuencias que incluirá las variables 'n', 'N', 'f(%)' y 'F(%)'. 'n' representa la frecuencia absoluta de cada compañía, mientras que 'N' es la frecuencia acumulada de estas frecuencias absolutas. 'f(%)' muestra la frecuencia relativa en porcentaje, indicando el porcentaje de veces que cada compañía aparece en relación al total de apariciones, y 'F(%)' es la frecuencia relativa acumulada en porcentaje. Utilizaremos un diccionario llamado 'tablafrec' para organizar estos datos y crearemos un DataFrame llamado 'df1' con 'pd.DataFrame(tablafrec, columns=['n', 'N', 'f(%)', 'F(%)'])' para especificar el orden de las columnas. Finalmente, imprimiremos 'df1' para visualizar esta información estructurada.

Ejecución:

	n	N	f(%)	F(%)
Apple	21	21	1.611665	1.611665
HP	274	295	21.028396	22.640061
Acer	103	398	7.904835	30.544896
Asus	158	556	12.125863	42.670760
Dell	297	853	22.793553	65.464313
Lenovo	297	1150	22.793553	88.257866
Chuwi	3	1153	0.230238	88.488104
MSI	54	1207	4.144282	92.632387
Microsoft	6	1213	0.460476	93.092863
Toshiba	48	1261	3.683807	96.776669
Huawei	2	1263	0.153492	96.930161
Xiaomi	4	1267	0.306984	97.237145
Vero	4	1271	0.306984	97.544129
Razer	7	1278	0.537222	98.081351
Mediacom	7	1285	0.537222	98.618573
Samsung	9	1294	0.690714	99.309286
Google	3	1297	0.230238	99.539524
Fujitsu	3	1300	0.230238	99.769762
LG	3	1303	0.230238	100.000000

Figura 20: Ejecución 11.

Fuente: Propia, echo en Google Colab.

Conclusiones:

En la primera parte del análisis de frecuencias ('n'), observamos que Dell y Lenovo son las marcas más frecuentes, cada una con 297 apariciones, lo que representa aproximadamente el 22.79% del total de marcas de laptops en nuestro conjunto de datos. HP sigue de cerca con 274 apariciones, representando un 21%, y Asus con un 12,13%. En contraste, las marcas menos comunes son Huawei, Xiaomi y Vero, con una presencia muy baja; Por ejemplo, Huawei representa solo el 0,15%. En conjunto, las marcas líderes (Dell, Lenovo, HP y Asus) constituyen aproximadamente el 65.46% de nuestro conjunto de datos, mientras que marcas como Chuwi, Google y Fujitsu cada una representan menos del 1% de nuestros datos.

Programación:

```
plot = datos_data['Ram'].hist(edgecolor='black', linewidth=1.2)
plot.set_title('RAM evaluadas')
plt.show()
```

Figura 20: Programación 12.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Comenzamos accediendo a la columna "Ram" de nuestro DataFrame utilizando 'datos_data['Ram']'. Luego, aplicamos el método 'hist()' para crear un histograma que represente la distribución de los valores en esa columna. Al utilizar 'edgecolor='black' y 'linewidth=1.2', establecemos el color y el grosor del borde de las barras del histograma para mejorar su visualización. Después, utilizamos 'plt.show()' para mostrar el histograma generado. Finalmente, podemos agregar un título al histograma utilizando 'plt.title()' para proporcionar una descripción clara del contenido. Este proceso nos permite visualizar de manera efectiva la distribución de la memoria RAM en nuestro conjunto de datos.

Ejecución:

Fuente: Propia, echo en Google Colab.

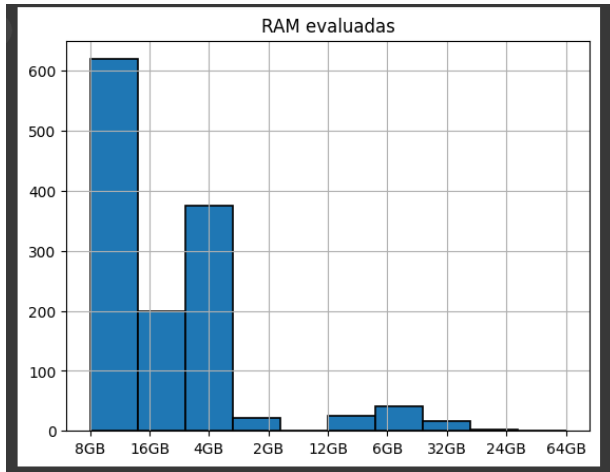


Figura 21: Ejecución 11.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El análisis del histograma de la columna "Ram" revela que la configuración más común entre los equipos es de 8GB, con más de 600 unidades registradas, seguida de cerca por la de 4GB, que cuenta con menos de 400. Esto sugiere que la mayoría de los usuarios prefieren una cantidad moderada de RAM. Sin embargo, a medida que aumenta la cantidad de RAM, la frecuencia de equipos con esa configuración disminuye, lo que indica una tendencia a no optar por configuraciones con mayores cantidades de RAM. Esta tendencia puede estar relacionada con el aumento de los costos asociados con configuraciones de memoria más grandes, lo que lleva a los usuarios a buscar un equilibrio entre el rendimiento y el costo al elegir una computadora portátil.

Programación:

```
fig = plt.figure(figsize=(10, 7))

# Creating plot
plt.boxplot(datos_data["Inches"])
plt.title("Boxplot Medidas en Pulgadas")
plt.grid()
#plt.xlabel('X (m)')
plt.ylabel('Inches')
# show plot
plt.show()
```

Figura 22: Programación 13.

Procedimientos:

Para generar el diagrama de caja, primero establecemos el tamaño del gráfico para garantizar su legibilidad. Luego, accedemos a la columna "Pulgadas" del DataFrame "data_datos" para obtener los datos de las pulgadas de los equipos. Utilizamos el método 'boxplot()' para crear el diagrama de caja que mostrará la distribución de estos datos. Para mejorar la presentación del gráfico, añadimos una cuadrícula con 'plt.grid()' y etiquetamos el eje y como "Pulgadas" con 'plt.ylabel("Pulgadas")'. Finalmente, mostramos el diagrama de caja con 'plt.show()', lo que nos permite examinar fácilmente la mediana, los cuartiles y cualquier valor atípico en la distribución de las pulgadas de los equipos.

Ejecución:

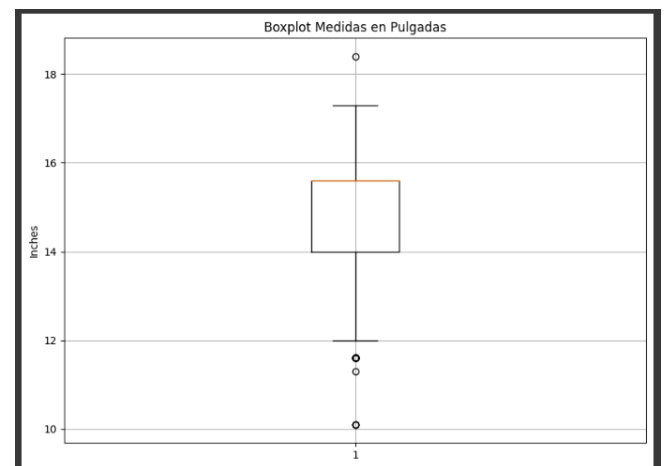


Figura 23: Ejecución 13.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La interpretación del diagrama de caja revela que la mayoría de los equipos tienen medidas de pantalla que oscilan entre las 15 y las 16 pulgadas, con la mayor concentración de datos entre las 12 y las 16 pulgadas. Además, se identifican valores atípicos tanto por encima de las 18 pulgadas como por debajo de las 10 pulgadas, lo que sugiere la presencia de equipos con tamaños de pantalla inusuales en nuestro conjunto de datos. Estos hallazgos son consistentes con la observación de la distribución de datos en la gráfica de barras de las pulgadas, donde el menor valor registrado es 10 pulgadas y el mayor es 18 pulgadas. La simetría en la distribución de los datos alrededor de la mediana sugiere una distribución relativamente equilibrada, con una

cantidad similar de datos tanto por encima como por debajo de este punto central.

Programación:

```
import pandas as pd
important_features = ['laptop_ID', 'Company', 'Product', 'TypeName', 'Inches', 'ScreenResolution', 'Cpu', 'Ram', 'Memory', 'Gpu', 'OpSys', 'Weight']
datos_data = datos.loc[:, important_features]
```

Figura 24: Programación 14.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para iniciar, importamos la biblioteca Pandas y la renombramos como 'pd', simplificando así su uso en nuestro código. Luego, creamos una lista llamada "important_features", que contiene los nombres de las columnas que consideramos esenciales en nuestro DataFrame de datos. Estas columnas pueden contener información crucial para nuestro análisis o para la construcción de modelos. Posteriormente, creamos un nuevo DataFrame llamado "datos_data" utilizando el método 'loc[]' de Pandas. Este método nos permite seleccionar las columnas específicas del DataFrame original, utilizando los nombres de las columnas contenidos en la lista "important_features". Al hacerlo, generamos un nuevo DataFrame que contiene únicamente las columnas que consideramos relevantes para nuestro análisis futuro. Este proceso de selección nos permite concentrarnos en las características más importantes de nuestros datos, lo que facilita la interpretación y el análisis de los mismos.

Programación:

```
#Se definen las clases
bins=[10,12,14,16,18]
labels=['10 - 12','12 - 14','14 - 16','16 - 18']
```

Figura 25: Programación 15.

Fuente: Propia, echo en Google Colab.

Procedimientos:

En la primera línea "bins=[10,12,14,16,18]" esta línea define los intervalos para agrupar los datos, cada número representa el límite superior de un intervalo. Entonces en el intervalo iría así de 10 (que no pertenece) hasta 12 (que no pertenece) y así con los demás datos. En la línea "labels=['10 - 12','12 - 14','14 - 16','16 - 18']" define las etiquetas o los nombres para cada intervalo correspondiente a la lista "bins" para obtener un rango donde va de 10 (no incluye) hasta 12.

Programación:

```
df['Clases']=pd.cut(datos_data['Inches'],bins,labels=labels)
```

Figura 26: Programación 15.

Fuente: Propia, echo en Google Colab.

Procedimientos:

La función 'pd.cut()' de Pandas para segmentar los datos de la columna "Pulgadas" en intervalos definidos por el parámetro 'bins', los cuales pueden ser rangos predeterminados o especificados manualmente. Luego, se asignan etiquetas a estos intervalos utilizando el parámetro 'labels'. En el código dado, esta operación se realiza en el DataFrame 'datos_data'. Una vez que los datos se han segmentado y etiquetado, el resultado se guarda en una nueva columna llamada "Clases" en el DataFrame 'df', lo que nos permite analizar los datos en términos de estas categorías discretas. Este enfoque es útil cuando se desea agrupar datos continuos en categorías discretas para un análisis más detallado o para facilitar el procesamiento de datos en modelos predictivos.

Programación:

```
tabla2=pd.crosstab(index=df['Clases'],columns='n')
tabla2
```

Figura 27: Programación 16.

Fuente: Propia, echo en Google Colab.

Procedimientos:

La función pd.crosstab() de Pandas para generar una tabla de frecuencias cruzadas, donde la columna "Clases" del DataFrame df se utiliza como el índice de la tabla. Al especificar columns='n', indicamos que queremos contar la frecuencia de las clases y almacenar los resultados en la variable tabla2. Esta tabla nos ofrece una visión estructurada de la distribución de las clases en nuestro conjunto de datos, mostrando cuántas veces aparece cada clase. La impresión posterior de la tabla tabla2 nos permite visualizar de manera clara y concisa estas frecuencias, lo que facilita la interpretación y el análisis de los datos.

Ejecución:

col_0	n
Inches	
10 - 12	44
12 - 14	419
14 - 16	674
16 - 18	165

Figura 28: Ejecución 16.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Comenzamos organizando los datos de la clase "Pulgadas" en cuatro rangos predefinidos: de 10 a 12 pulgadas, de 12 a 14 pulgadas, de 14 a 16 pulgadas y de 16 a 18 pulgadas. Utilizando la función `pd.crosstab()`, contamos las veces que se repiten los datos en cada uno de estos rangos y almacenamos los resultados en la columna "n". Al analizar esta tabla, podemos observar que los rangos de 14 a 16 pulgadas son los más frecuentes, lo que sugiere que la mayoría de los equipos tienen tamaños de pantalla dentro de este intervalo.

Programación:

```
tabla2['N']=tabla2['n'].cumsum()
tabla2['f']=100*tabla2['n']/sum(tabla2['n'])
tabla2['F']=tabla2['f'].cumsum()
tabla2
```

Figura 29: Programación 17.

Fuente: Propia, echo en Google Colab.

Procedimientos:

En nuestra primera línea `tabla2['N']=tabla2['n'].cumsum()` calcula la frecuencia acumulada de nuestra clase asignada y la asigna a una nueva columna llamada "N" utilizando un método `cumsum()` para así calcular la suma acumulada de "n". Ahora con `tabla2['f']=100*tabla2['n']/sum(tabla2['n'])` se calcula la frecuencia relativa de cada clase y se almacena en una nueva columna llamada "f". Se divide la frecuencia absoluta de cada clase "n" por el total de frecuencia absolutas de todas las clases lo podemos ver en `sum(tabla2['n'])` para luego ser

multiplicado por 100 y obtener así un porcentaje. Por último `tabla2['F']=tabla2['f'].cumsum()` calculamos la frecuencia relativa acumulada y se le asigna a la variable "F", utilizamos `cumsum()` para calcular la suma acumulativa de las frecuencias relativas "f".

Ejecución:

col_0	n	N	f	F
Inches				
10 - 12	44	44	3.379416	3.379416
12 - 14	419	463	32.181260	35.560676
14 - 16	674	1137	51.766513	87.327189
16 - 18	165	1302	12.672811	100.000000

Figura 30: Ejecución 17.

Fuente: Propia, echo en Google Colab.

Conclusiones:

En el análisis de las clases de tamaño de pantalla de laptops, se observa una concentración significativa en las clases de 12 a 14 pulgadas y de 14 a 16 pulgadas, que representan conjuntamente aproximadamente el 83.95% de todas las medidas de laptops en nuestro conjunto de datos. Dentro de estas clases, la clase de 14 a 16 pulgadas es la más frecuente, representando casi el 51.77% de todas las medidas, lo que indica que este rango de tamaño de pantalla es el más preferido por los consumidores. Por otro lado, las clases de 10 a 12 pulgadas, que representan las pantallas más pequeñas en nuestros datos, son menos comunes, a cubrir solo el 3.38% del total. Estas estadísticas sugieren que es más probable que los usuarios compren laptops con pantallas de 14 a 16 pulgadas, lo que puede deberse a preferencias de comodidad visual o necesidades de rendimiento.

Programación:

```
df = datos_data['Weight'].value_counts()
print(df)
```

Figura 31: Programación 18.

Fuente: Propia, echo en Google Colab.

Procedimientos:

En nuestra primera línea “df = datos_data['Weight'].value_counts()” esta línea cuanta la cantidad de apariciones de cada valor único en la columna “Weight” de nuestro DataFrame “datos_data” donde asignara estos conteos a “df” para después ser mostrada.

Ejecución:

```
2.2kg 121
2.1kg 58
2.4kg 44
2.3kg 41
2.5kg 38
...
3.25kg 1
4.7kg 1
1.55kg 1
1.18kg 1
4.0kg 1
Name: Weight, Length: 179, dtype: int64
```

Figura 32: Ejecución 18.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El análisis de las frecuencias de los pesos de las laptops revela que los valores más comunes se encuentran en el rango de 2.2kg a 2.5kg. Esto sugiere que existe una preferencia generalizada entre los consumidores por modelos que no sean ni demasiado livianos ni demasiado pesados. Sin embargo, los pesos de los dispositivos pueden variar ligeramente debido a diferentes especificaciones y materiales de construcción. Por lo tanto, es más conveniente organizar y analizar estos datos en rangos, lo que nos permite evaluar cuántos equipos caen dentro de cada intervalo de peso y obtener una visión más clara de las preferencias de los usuarios en cuanto al peso de las laptops.

Programación:

```
data=datos_data['Weight']
n=data.value_counts(sort=False)
N=n.cumsum()
f=100*data.value_counts(sort=False)/len(data)
F=100*f.cumsum()/f.sum()

#Crear el diccionario y el dataframe de la tabla de frecuencias
tablafrec={'n':n,'N':N,'f(%)':f,'F(%)':F}
df1 = pd.DataFrame(tablafrec,columns=['n','N','f(%)','F(%)'])
print(df1)
```

Figura 33: Programación 19.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para generar una tabla de frecuencias de los pesos de las laptops, comenzamos extrayendo la columna "Weight" del DataFrame "datos_data" para obtener los datos de peso. Luego, calculamos la frecuencia absoluta, representada como "n", mediante el método `value\_counts()`, que cuenta las ocurrencias de cada valor único en la columna "Weight". Posteriormente, obtenemos la frecuencia absoluta acumulativa, "N", sumando acumulativamente las frecuencias absolutas. Continuamos calculando las frecuencias relativas, que representan el porcentaje de cada valor de peso con respecto al número total de observaciones en la columna "Weight". Estas frecuencias se expresan en porcentaje multiplicando las frecuencias absolutas por 100 y dividiéndolas por la longitud total de la columna "Weight". Finalmente, para obtener la frecuencia relativa acumulada, sumamos acumulativamente las frecuencias relativas y dividimos el resultado por la suma total de las frecuencias relativas. Con estos cálculos completos, tenemos todos los datos necesarios para organizar y construir nuestra tabla de frecuencias de los pesos de las laptops.

Procedimientos:

el código calcula una tabla de frecuencias para los pesos de las laptops, a empaquetar desde 1.37 kg hasta 4.0 kg, que son el menor y el mayor valor respectivamente. La columna "n" muestra el número de frecuencias correspondiente a cada peso, revelando que los valores más comunes son 2.1 kg y 2.2 kg, con frecuencias que representan el 9% y el 4% respectivamente. Esto indica que estos pesos son los más comunes entre las laptops en el conjunto de datos. Por el contrario, el peso de 4,0 kg solo se repite una vez, lo que demuestra su rareza en comparación con los pesos más comunes como 2,2 kg o 2,34 kg.

Programación:


```
import pandas as pd
important_features = ['laptop_ID', 'Company', 'Product', 'Type', 'Inches', 'ScreenResolution']
datos_data = datos.loc[:, important_features]
```

Figura 34: Programación 20.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para comenzar importamos nuestro “panda as pd” después de esto guardaremos todas nuestras columnas de nuestro DataFrame y se guardaran en “important_freatures” de la siguiente forma “important_features = ['laptop_ID', 'Company', 'Product', 'Type', 'Inches', 'ScreenResolution', 'Cpu', 'Ram', 'Memory', 'Gpu', 'OpSys', 'Weight', 'Price_euros']”. Ahora usando el método “loc” de nuestra librería importada seleccionara todas las filas y solo las columnas específicas de la lista “important_features”.

Programación:

```
#Se definen las clases
bins=[0,1000,2000,3000,4000,5000,6000,7000,8000,9000,10000]
labels=['0 - 1000', '1000 - 2000', '2000 - 3000', '3000 - 4000', '4000 - 5000', '5000 - 6000', '6000 - 7000', '7000 - 8000', '8000 - 9000', '9000 - 10000']
```

Figura 35: Programación 20.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para la siguiente parte del código se definirán los límites de los intervalos de clases en una lista llamada “bins” nuestro primer intervalo va desde 0 a 1000 y así con nuestros siguientes valores. Para la siguiente parte definiremos los intervalos de la clase, esto permitirá identificar cada intervalo después de usar el método “binning”.

Programación:

```
df['Clases']=pd.cut(datos_data['Price_euros'],bins,labels=labels)
```

Figura 36: Programación 21.

Fuente: Propia, echo en Google Colab.

Procedimientos:

En la siguiente parte “df['Clases']=pd.cut(datos_data['Price_euros'],bins,labels=labels)” en nuestra primera parte “datos_data” almacena los precios en euros, nuestro “bins” y “labels” contienen nuestros limites

de intervalos de clases. Después de esto se crea una nueva columna llamada “Clases” en el DataFrame “df”.

Programación:

```
tabla2=pd.crosstab(index=df['Clases'],columns='n')
tabla2
```

Figura 37: Programación 22.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para terminar usaremos “crosstab()” que nos ayuda a hacer una tabla de frecuencias, después de esto accedemos a la columna “Clases” y nuestras columnas “n” que representara el número de frecuencias observadas y con “index” especifica las filas de las tablas de frecuencias. Todo esto se guardará en “tabla2” que se imprimirá después.

Ejecución:

col_0	n
Price_euros	
0 - 1000	683
1000 - 2000	483
2000 - 3000	118
3000 - 4000	15
4000 - 5000	2
5000 - 6000	1
6000 - 7000	1

Figura 38: Ejecución 22.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El precio que es más común en invertir en un computador están entre los 0 a 1000 euros esto ya que con este precio se pueden encontrar computadores con buenas referencias a diferencia de 6000 a 7000 euros que este computador es de la

marca “Razer” este es el precio más elevado entre todos los datos también lo pudimos verificar en nuestro diagrama de Boxplot.

Programación:

```
tabla2['N']=tabla2['n'].cumsum()
tabla2['f']=100*tabla2['n']/sum(tabla2['n'])
tabla2['F']=tabla2['f'].cumsum()
tabla2
```

Figura 39: Programación 23.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para crear nuestra tabla de frecuencias utilizaremos “tabla2['N'] = tabla2['n'].cumsum()” esta calculara la suma acumulativa de las frecuencias absolutas y se creara una nueva columna llamada “N”. Ahora calcularemos la frecuencia relativa por medio de “tabla2['f'] = 100*tabla2['n']/sum(tabla2['n'])” por medio del total de las frecuencias “n” para terminar “sum(tabla2['n'])” se utiliza como denominador para calcular la frecuencia relativa de cada intervalo. Para la frecuencia acumulada relativa de los intervalos de clase, tomamos la columna de las frecuencias relativas “f” para así después ir sumando acumuladamente y guardarla en la variable “F”.

Ejecución:

col_0	n	N	f	F
Price_euros				
0 - 1000	683	683	52.417498	52.417498
1000 - 2000	483	1166	37.068304	89.485802
2000 - 3000	118	1284	9.056025	98.541827
3000 - 4000	15	1299	1.151190	99.693016
4000 - 5000	2	1301	0.153492	99.846508
5000 - 6000	1	1302	0.076746	99.923254
6000 - 7000	1	1303	0.076746	100.000000

Figura 40: Ejecución 23.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El código realiza un análisis de la distribución de precios de laptops, dividiendo los datos en diferentes rangos de precios y calculando el porcentaje de laptops en cada rango. Se observa que más de la mitad de los resultados se encuentran en el rango de precios de 0 a 1000 euros, representando el 52,4% del total de datos. Le sigue el rango de 1000 a 2000 euros, que abarca el 37,06% de las portátiles en el conjunto de datos. Estos dos rangos en conjunto suman el 84% de los datos, lo que indica que la mayoría de las computadoras portátiles tienen precios relativamente bajos o moderados. Por otro lado, los rangos de precios más altos, como 4000, 5000 y 6000 euros, no alcanzan ni siquiera el 1% de las portátiles, lo que sugiere que los modelos más caros son menos comunes en el conjunto de datos. Este análisis proporciona una visión clara de la distribución de precios de las laptops y destaca las tendencias predominantes en términos de asequibilidad y disponibilidad de modelos en diferentes rangos de precios.

Programación:

```
plot = df['Clases'].hist(edgecolor='black', linewidth=1.2)
plot.set_title("Precios en Euros por clases [€]")
plt.xlabel('€')
plt.show()
```

Figura 41: Programación 24.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Ahora crearemos el histograma, primero accederemos a la columna “Clases” de nuestro DataFrame de “df” que contiene los intervalos de los precios de los portátiles. Por medio de “.hist()” este método de Pandas creara un histograma de los datos en la columna ya especificada. Para finalizar estableceremos las medidas, color, titulo con el símbolo de euros.

Ejecución:

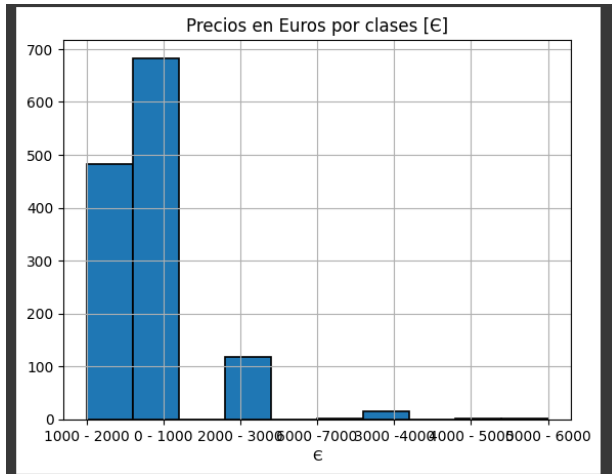


Figura 42: Ejecución 24.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Las barras más altas corresponden a los rangos de 0 a 1000 euros y de 1000 a 2000 euros, lo que indica que la mayoría de las portátiles tienen precios dentro de estos dos rangos. A continuación, la barra para el rango de 2000 a 3000 euros muestra un número considerable de portátiles, con 118 unidades representadas. Sin embargo, los puntos más bajos en el gráfico corresponden a los rangos de precios más altos, de 5000 a 6000 euros y de 6000 a 7000 euros, que apenas sobresalen. Esto sugiere que las laptops con precios más altos son menos comunes en el conjunto de datos, con una presencia significativamente menor en comparación con los rangos de precios más bajos y moderados.

Programación:

```
import matplotlib.pyplot as plt

# Get the company names and their counts
company_names = datos_data['Company'].value_counts().index.to_list()
company_counts = datos_data['Company'].value_counts().to_list()

# Create a pie chart
plt.figure(figsize=(10, 7))
plt.pie(company_counts, labels=company_names, autopct='%1.1f%%')
plt.title("Distribución de laptops por compañía")
plt.show()
```

Figura 43: Programación 25.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para comenzar importamos la biblioteca Matplotlib esta tiene la finalidad de trazar gráficos. En la siguiente línea "company_names" = datos_data['Company'].value_counts().index.to_list() se toma los datos de "datos_data" y la columna "Company" devolviendo una serie que contiene los nombres de la compañías como índices y el recuento que tienen luego de esto se agregara a "company_counts" mientras los índices o nombres de compañías se agregaran a "company_names" Para el grafico se creara una nueva figura con un tamaño en específico. Después va agregar la lista de "company_names", la lista de "company_names" que usara como etiquetas cada porción del gráfico y para terminar "autopct='%1.1f%%'" que mostrara en el grafico el porcentaje con decimales. Solo faltaria ponerle el titulo y mostrarlo.

Ejecución:

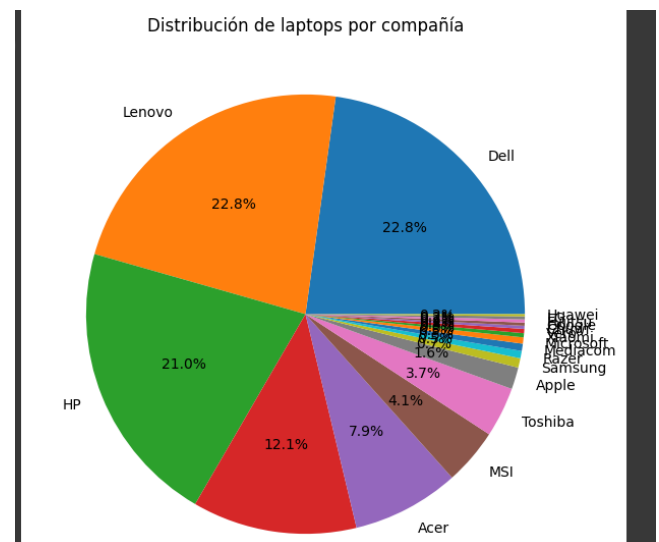


Figura 44: Ejecución 25.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La concentración de datos en las tres principales compañías: Lenovo, Dell y HP. Estas tres empresas representan más del 65% del conjunto de datos, lo que indica una fuerte presencia en el mercado de portátiles. La proporción significativa de datos atribuidos a estas tres compañías sugiere que son las marcas más populares o ampliamente utilizadas entre los usuarios de computadoras portátiles. Por otro lado, el hecho de que el resto de las compañías comprende solo el 35% de los datos refleja una diversidad más amplia en el mercado, con 13

el recuento que tienen luego de esto se agregara a “company_counts” mientras los índices o el tipo de resolución de la tarjeta se agregara a “company_names” Para el grafico se creara una nueva figura con un tamaño en específico. Después va agregar la lista de “company_names”, la lista de “company_names” que usara como etiquetas cada porción del gráfico y para terminar “autopct=\"%1.1f%%” que mostrara en el grafico el porcentaje con decimales. Solo faltaria ponerle el titulo y mostrarlo. Ejecución:

```
import matplotlib.pyplot as plt

# Get the company names and their counts
company_names = datos_data['OpSys'].value_counts().index.to_list()
company_counts = datos_data['OpSys'].value_counts().to_list()

# Create a pie chart
plt.figure(figsize=(10, 7))
plt.pie(company_counts, labels=company_names, autopct=\"%1.1f%%\")
plt.title(\"Sistemas operativos de las laptop\")
plt.show()
```

Figura 47: Programación 28.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para comenzar importamos la biblioteca Matplotlib esta tiene la finalidad de trazar gráficos como lo vimos en el anterior código. En la siguiente línea “company_names = datos_data[“OpSys”].value_counts().index.to_list()” se toma los datos de “datos_data” y la columna “OpSys” devolviendo una serie que contiene los nombres de los sistemas operativos como índices y el recuento que tienen luego de esto se agregara a “company_counts” mientras los índices o el tipo de sistema se agregara a “company_names” Para el grafico se creara una nueva figura con un tamaño en específico. Después va agregar la lista de “company_names”, la lista de “company_names” que usara como etiquetas cada porción del gráfico y para terminar “autopct=\"%1.1f%%” que mostrara en el grafico el porcentaje con decimales. Solo faltaria ponerle el titulo y mostrarlo. Ejecución:

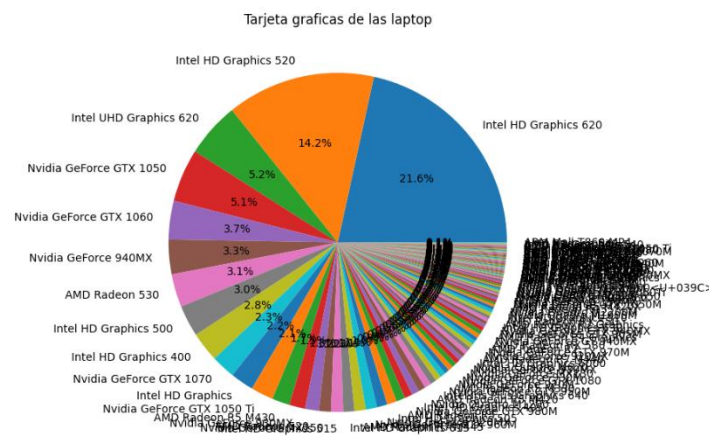
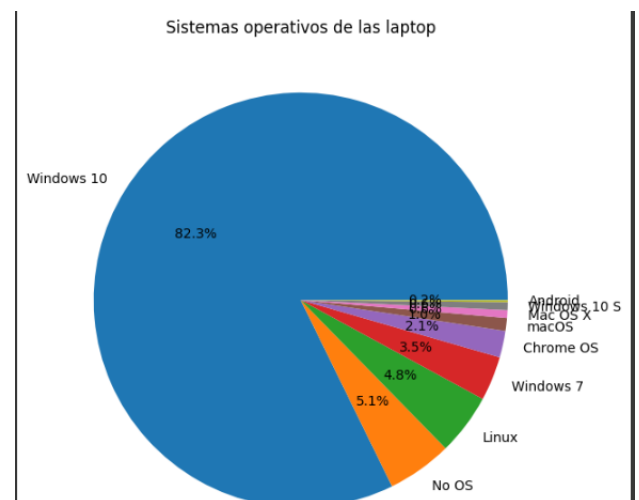


Figura 46: Ejecución 27.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El análisis de los tipos de procesadores revela una amplia variedad en el mercado, con múltiples marcas compitiendo por participación. Intel destaca como el líder dominante, representando casi el 39% del mercado de procesadores. Sin embargo, a medida que se observa la distribución más detallada, se nota que Intel no solo es la marca más popular, sino que también tiene una presencia significativa a lo largo de la gráfica, lo que indica una amplia gama de modelos y versiones disponibles para los consumidores. Por otro lado, aunque las juntas Nvidia y AMD abarcan alrededor del 64% del mercado, gran parte de este porcentaje está impulsado por Nvidia, mientras que AMD contribuye con diversas versiones. Esto sugiere que, aunque algunas versiones específicas de Intel pueden ser más populares, la variedad ofrecida por otras marcas, como Nvidia y AMD, juega un papel importante en la elección de los consumidores. En resumen, mientras que Intel domina en términos de participación de mercado, la diversidad de marcas y versiones proporciona a los consumidores una variedad de opciones para elegir, lo que influye en la distribución general de los procesadores. Programación:

Figura 48: Ejecución 28.

Fuente: Propia, echo en Google Colab.

Conclusiones:

A simple vista se puede ver que Window 10 domina el mercado siendo el más común en los computadores ya que este es muy compatible con el software y la facilidad de uso es confortable para muchos usuarios. Donde hay presencia minoritaria es en “macOS” que solo tiene el 2.1% siendo la segunda opción más popular ya podeos ver que Linuz y Windows 7 también están, este Windows es usado ya que muchos usuarios no han actualizado a Windows 10 o que sus computadores no soporten esta actualización.

Programación:

```
data=datos_data['Memory']
n=data.value_counts(sort=False)
N=n.cumsum()
f=100*data.value_counts(sort=False)/len(data)
F=100*f.cumsum()/f.sum()

#Crear el diccionario y el dataframe de la tabla de frecuencias
tablafrec={'n':n,'N':N,'f(%)':f,'F(%)':F}
df1 = pd.DataFrame(tablafrec,columns=['n','N','f(%)','F(%)'])
print(df1)
```

Figura 49: Programación 29.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Ahora realizaremos una tabla de frecuencias de las memorias. Primero llamaremos a nuestro “datos_data” extrayendo en el la columna de “Memory”. Ahora encontraremos nuestro “n” o la frecuencia absoluta, buscaremos los índices con los valores más frecuentes de esta manera “n=data.value_counts(sort=False)”. Teniendo esto encontremos nuestra frecuencia absoluta acumulativa “N=n.cumsum()” estos datos se guardarán en N. Ahora seguiremos con las frecuencias, la frecuencia relativa se calcula “f=100*data.value_counts(sort=False)/len(data)” mediante cada valor de nuestro “data” encontrara el porcentaje del número total de observaciones. Para terminar nos faltaría la frecuencia relativa acumulada “F=100*f.cumsum()/f.sum()” con cada valor único de “data” con un porcentaje en total. En la siguiente parte llamaremos nuestras variables que contienen lo necesario para la tabla de frecuencias teniendo en cuenta que las

frecuencias están dadas en porcentajes, para después ser organizadas.

Ejecución:

	n	N	f(%)	F(%)
128GBSSD	76	76	5.832694	5.832694
128GB Flash Storage	4	80	0.306984	6.139678
256GBSSD	412	492	31.619340	37.759018
512GBSSD	118	610	9.056025	46.815042
500GBHDD	132	742	10.130468	56.945510
256GB Flash Storage	8	750	0.613968	57.559478
1TBHDD	223	973	17.114351	74.673830
32GB Flash Storage	38	1011	2.916347	77.590177
128GBSSD + 1TBHDD	94	1105	7.214121	84.804298
256GBSSD + 256GBSSD	2	1107	0.153492	84.957790
64GB Flash Storage	15	1122	1.151190	86.108979
256GBSSD + 1TBHDD	73	1195	5.602456	91.711435
256GBSSD + 2TBHDD	10	1205	0.767460	92.478895
32GBSSD	6	1211	0.460476	92.939371
2TBHDD	16	1227	1.227936	94.167306
64GBSSD	1	1228	0.076746	94.244052
1.0TB Hybrid	9	1237	0.690714	94.934766
512GBSSD + 1TBHDD	14	1251	1.074444	96.009210
1TBSSD	14	1265	1.074444	97.083653
256GBSSD + 500GBHDD	2	1267	0.153492	97.237145
128GBSSD + 2TBHDD	2	1269	0.153492	97.390637
512GBSSD + 512GBSSD	1	1270	0.076746	97.467383
16GBSSD	3	1273	0.230238	97.697621
16GB Flash Storage	7	1280	0.537222	98.234843
512GBSSD + 256GBSSD	1	1281	0.076746	98.311589
512GBSSD + 2TBHDD	3	1284	0.230238	98.541827
64GB Flash Storage + 1TBHDD	1	1285	0.076746	98.618573
180GBSSD	5	1290	0.383730	99.002302
1TBHDD + 1TBHDD	1	1291	0.076746	99.079048
32GBHDD	1	1292	0.076746	99.155794
1TBSSD + 1TBHDD	2	1294	0.153492	99.309286
512GB Flash Storage	2	1296	0.153492	99.462778
128GBHDD	1	1297	0.076746	99.539524
240GBSSD	1	1298	0.076746	99.616270
8GBSSD	1	1299	0.076746	99.693016
508GB Hybrid	1	1300	0.076746	99.769762
1.0TBHDD	1	1301	0.076746	99.846508
512GBSSD + 1.0TB Hybrid	1	1302	0.076746	99.923254
256GBSSD + 1.0TB Hybrid	1	1303	0.076746	100.000000

Figura 50: Ejecución 29.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La mayoría de los laptops usan unidades de estado sólido (SSD) como medida de almacenamiento principal porque se sabe que estas son más efectivas a la hora de almacenar información y rapidez estas se encuentran en 256GB SSD y 512GB SSD siendo estas las más comunes representando

aproximadamente el 31.6% y el 9.1%, el SSD se encuentra casi en el 50%. También el (HDD) unidades de disco duro son muy comunes muchas veces estas son las más comunes a la hora de comprar un computador, pero este depende del uso que quieras darle tu computador. El HDD se encuentra en 1TB siendo este la más frecuente con un 17.1%.

Programación:

```
import numpy as np

mean = np.mean(datos_data['Inches'])
median = np.median(datos_data['Inches'])
q1 = np.percentile(datos_data['Inches'], 25)
q3 = np.percentile(datos_data['Inches'], 75)

print("Media:", mean)
print("Mediana:", median)
print("Q1:", q1)
print("Q3:", q3)
```

Figura 51: Programación 30.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Se importa la biblioteca NumPy. Para calcular la media de los datos de las pulgadas de las pantallas “np.mean(datos_data['Inches'])”, la media es simplemente el promedio de todos los valores en la columna 'Inches' esto lo sacamos por medio de nuestra biblioteca con “mean” que nos ayuda a hallar este valor. La mediana se calcula en esta parte del código “median = np.median(datos_data['Inches'])” donde volveremos a usar la biblioteca y con ayuda de “median” se hallara este resultado que significa el centro de un conjunto de datos ordenados. El primer cuartil se halla por “q1 = np.percentile(datos_data['Inches'], 25)” por medio de “percentile” e indicándole que es el 25% que este porcentaje hace referencia a nuestro primer cuartil. Lo mismo se hará con nuestro tercer percentil la diferencia es que indicaremos que es el 75% el segundo percentil es la media o el 50%. Ejecución:

```
Moda: 15.017191097467379
Mediana: 15.6
Q1: 14.0
Q3: 15.6
```

Figura 52: Ejecución 30.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El tamaño promedio de las pantallas esta entre las 15.02 esto se puede ver en el diagrama de Boxplot y el diagrama de barras que muestran que el tamaño más común es este. La mediana nos indica que el valor que esta la mitad es 15.6, esto nos indica que al menos la mitad de los laptops tiene un tamaño igual o menor a este lo cual es correcto ya que el valor más común es 15.02 entrando en este límite. Nuestro primer cuartil es de 14.0 pulgadas diciendo así que el 25% de los laptops tiene una pantalla tiene un tamaño igual o inferior, esto es verdadero ya que en el diagrama de barras se puede ver que las demás barras se encuentran por debajo de 14.0. Nuestro tercer cuartil es de 75% y es de 15.6 indica que los laptops tienen un tamaño igual o inferior lo cual tiene sentido teniendo en cuenta que nuestro dato más común es de 15.02.

Programación:

```
import numpy as np

mean = np.mean(datos_data['Price_euros'])
median = np.median(datos_data['Price_euros'])
q1 = np.percentile(datos_data['Price_euros'], 25)
q3 = np.percentile(datos_data['Price_euros'], 75)

print("Media:", mean)
print("Mediana:", median)
print("Q1:", q1)
print("Q3:", q3)
```

Figura 53: Programación 31.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Se importa la biblioteca NumPy. Para calcular la media de los precios “np.mean(datos_data['Price_euros'])”, la media es simplemente el promedio de todos los valores en la columna 'Inches' esto lo sacamos por medio de nuestra biblioteca con “mean” que nos ayuda a hallar este valor. La mediana se calcula en esta parte del código “median = np.median(datos_data['Price_euros'])” donde volveremos a usar la biblioteca y con ayuda de “median” se hallara este resultado que significa el centro de un conjunto de datos ordenados. El primer cuartil se halla por “q1 = np.percentile(datos_data['Price_euros'], 25)”

por medio de "percentile" e indicándole que es el 25% que este porcentaje hace referencia a nuestro primer cuartil. Lo mismo se hará con nuestro tercer percentil la diferencia es que indicaremos que es el 75% el segundo percentil es la media o el 50%.

Ejecución:

```
Moda: 1123.6869915579432
Mediana: 977.0
Q1: 599.0
Q3: 1487.88
```

Figura 54: Ejecución 31.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Nuestra media está en 1123 euros también vimos este resultado en el diagrama de Boxplot. La media es de 977 esto indica que la distribución de los datos es asimétrica a la derecha esto ocurre cuando tenemos valores extremadamente altos en un junto de datos. Nuestro primer cuartil es de 599 indicando esto e los laptops tienen un precio igual o inferior a este valor. Nuestro tercer cuartil es de 1487 significa que el 75% de los laptops tienen un precio igual o inferior a este valor, esto se puede notar en la concentración de valores entre los 1000 euros a 2000 euros.

Conclusiones finales del primer corte:

El análisis de los precios y tamaños de pantalla de las computadoras portátiles revela una variabilidad considerable en ambos aspectos. La media de los precios (1123,69 euros) es notable mayor que la mediana (977 euros), indicando la influencia de valores atípicos elevados. Un análisis más detallado muestra que los precios más comunes se concentran en el rango de 2700 a 3000 euros, aunque hay modelos que alcanzan hasta 6000 euros. "La variabilidad en los precios de las computadoras portátiles es notable, con un rango que abarca desde modelos económicos hasta dispositivos premium que pueden costar varias millas de euros" (TechRadar, 2023). En cuanto al tamaño de la pantalla, la mayoría de las portátiles tienen un tamaño promedio de 15,02 pulgadas, con una preferencia significativa por las de 15 pulgadas, que representan el 40% de las elecciones de los consumidores. "El tamaño de la pantalla de las computadoras portátiles es un factor determinante para muchos consumidores, con una clara preferencia por las pantallas de 15 pulgadas debido a su equilibrio entre portabilidad y funcionalidad" (PCMag, 2023). Existe una estimación positiva entre el tamaño de la pantalla y el precio, sugiriendo que los portátiles más grandes tienden a ser más caros. "Los modelos más caros tienden a ofrecer pantallas más grandes y de mayor calidad, así como

especificaciones técnicas superiores, lo que justifica en parte la variación en los precios" (Laptop Mag, 2023).

El conjunto de datos, organizado y tipificado adecuadamente, permite una vista más manejable y representativa para el análisis. La dispersión de los precios y la variabilidad en los tamaños de pantalla proporcionan una comprensión clara de las opciones disponibles y las preferencias del mercado. "La media de los precios de las computadoras portátiles puede estar significativamente influenciada por modelos de alta gama, que a menudo presentan características avanzadas y materiales premium" (CNET, 2023). La preparación de los datos sienta una base sólida para futuros análisis, facilitando la exploración de las relaciones entre las diferentes variables y la identificación de patrones en las decisiones de compra de los consumidores. Para la mayoría de los usuarios, un portátil de 15 pulgadas con un precio alrededor de la mediana (977 euros) es una opción recomendable, ya que ofrece un buen equilibrio entre tamaño de pantalla, rendimiento y costo, siendo ideal tanto para su uso personal como profesional sin incurrir en gastos excesivos. "Para la mayoría de los consumidores, una computadora portátil de 15 pulgadas que cuesta alrededor de 1000 euros ofrece una combinación ideal de precio, rendimiento y características, adecuada tanto para uso personal como profesional" (Consumer Reports, 2023).

Corte 2:

Para el segundo corte de nuestro estudio, enfocaremos nuestro análisis en la creación de modelos de regresión lineal y no lineal para determinar cuál se ajusta mejor a nuestro conjunto de datos, con el objetivo de evaluar cómo influye la generación del procesador ('processor_gnrtn'). en el precio de las computadoras en un país del tercer mundo. Este cambio de enfoque geográfico es fundamental para ajustar mejor los precios en un contexto donde la denominación monetaria y las condiciones económicas son significativamente diferentes a las de un país desarrollado. Además, analizamos las distribuciones y relaciones de variables como "ratings y tendencia", "ratings vs precio" y "frecuencia" mediante histogramas, para obtener una visión más detallada del mercado en este nuevo contexto.

Programación:

```
import pandas as pd
Datos = pd.read_csv("/content/drive/MyDrive/Datos/.ipynb_checkpoint")
Datos.head(1002)
```

Figura 54: Programación 32.

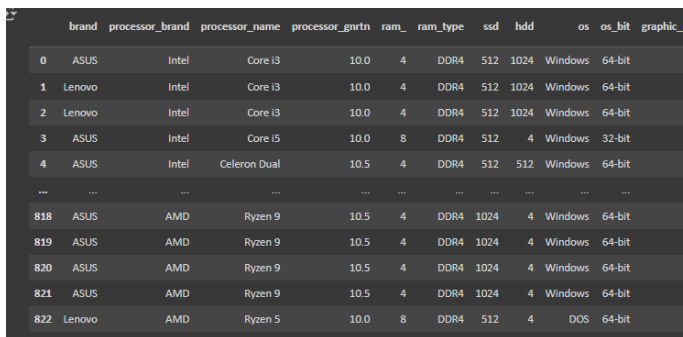
Fuente: Propia, echo en Google Colab.

Procedimientos:

En Google Colab, una plataforma de desarrollo en la nube basada en Python, la importación de la biblioteca "drive" y el montaje de Google Drive mediante `drive.mount('/content/drive')` facilitan el acceso a archivos almacenados en la nube desde el entorno de Colab. Este proceso permite que los usuarios trabajen con conjuntos de datos alojados en Google Drive de manera eficiente. Una vez montado, se asigna una variable, denominada DataFrame en el contexto de la biblioteca pandas, para representar los datos. Este DataFrame permite la manipulación y análisis de los datos de manera efectiva.

Posteriormente, al utilizar `data.head(1002)`, se muestran las primeras 1002 filas del DataFrame "data", proporcionando una vista previa de los datos y permitiendo comprender su estructura y contenido.

Ejecución:



	brand	processor_brand	processor_name	processor_gnrtn	ram_	ram_type	ssd	hdd	os	os_bit	graphic_c
0	ASUS	Intel	Core i3	10.0	4	DDR4	512	1024	Windows	64-bit	
1	Lenovo	Intel	Core i3	10.0	4	DDR4	512	1024	Windows	64-bit	
2	Lenovo	Intel	Core i3	10.0	4	DDR4	512	1024	Windows	64-bit	
3	ASUS	Intel	Core i5	10.0	8	DDR4	512	4	Windows	32-bit	
4	ASUS	Intel	Celeron Dual	10.5	4	DDR4	512	512	Windows	64-bit	
...	...	...	...	...	...	...	...	...	...	...	...
818	ASUS	AMD	Ryzen 9	10.5	4	DDR4	1024	4	Windows	64-bit	
819	ASUS	AMD	Ryzen 9	10.5	4	DDR4	1024	4	Windows	64-bit	
820	ASUS	AMD	Ryzen 9	10.5	4	DDR4	1024	4	Windows	64-bit	
821	ASUS	AMD	Ryzen 9	10.5	4	DDR4	1024	4	Windows	64-bit	
822	Lenovo	AMD	Ryzen 5	10.0	8	DDR4	512	4	DOS	64-bit	

Figura 55: Ejecución 32.

Fuente: Propia, echo en Google Colab.

Programación:



Figura 56: Programación 33.

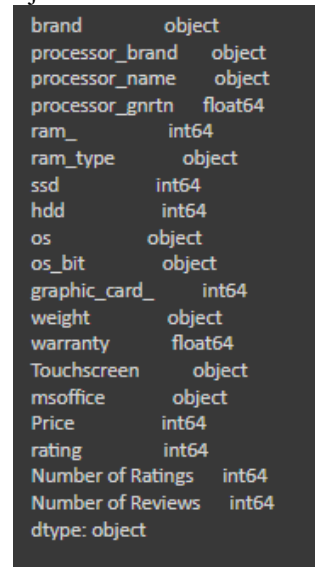
Fuente: Propia, echo en Google Colab.

Procedimientos:

Este código realiza una inspección detallada de los tipos de datos presentes en cada columna del conjunto de datos, que se llama "datos". Utilizando la función `dtypes`, se obtiene información sobre el tipo de variable presente en cada columna, lo que incluye datos como el nombre del producto, la compañía fabricante y otros atributos relevantes de las laptops. Esta inspección es crucial para comprender la naturaleza de los datos

con los que se está trabajando, lo que permite realizar análisis y manipulaciones específicas de acuerdo con los tipos de datos presentes en el conjunto.

Ejecución:



brand	object
processor_brand	object
processor_name	object
processor_gnrtn	float64
ram_	int64
ram_type	object
ssd	int64
hdd	int64
os	object
os_bit	object
graphic_card_	int64
weight	object
warranty	float64
Touchscreen	object
msoffice	object
Price	int64
rating	int64
Number of Ratings	int64
Number of Reviews	int64
dtype:	object

Figura 57: Ejecución 33.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Se observa el tipo de datos que se tienen en el DataSet, los cuales son flotantes, enteros y tipo String; lo que hace que el data set sea perfecto de analizar por su variabilidad de tipos de datos

Programación:

```
[ ] import pandas as pd

pima = pd.read_csv("/content/drive/MyDrive/Datos/.ipynb_checkpoints/laptopPrice.csv", encoding='la
pima.describe()
```

Figura 58: Programación 34.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Como se puede observar lo primero que hacemos es importar la biblioteca "drive" y con "drive.mount('/content/drive') nos permite acceder a los archivos almacenados de Google Drive pero desde Colab. Ya con esto, nombramos la variable de datos que va tomar el DataFrame, nos ayudara para poder mostrar las primeras 1002 filas del DataFrame. Finalmente generamos "data.describe()" que como hemos visto en clase nos ayudara para poder hallar estadísticas sobre el DataFrame.

Ejecución:

	processor_gnrtn	ram_	ssd	hdd	graphic_card_	warranty
count	823.000000	823.000000	823.000000	823.000000	823.000000	823.000000
mean	10.402795	8.680437	529.263670	230.089915	3.995140	1.183597
std	0.822230	4.372408	253.025214	411.444487	0.882363	0.236326
min	4.000000	4.000000	128.000000	4.000000	2.000000	1.000000
25%	10.000000	4.000000	512.000000	4.000000	4.000000	1.000000
50%	10.500000	8.000000	512.000000	4.000000	4.000000	1.000000
75%	11.000000	8.000000	512.000000	4.000000	4.000000	1.500000
max	12.000000	32.000000	3072.000000	2048.000000	8.000000	1.500000

Figura 59: Ejecución 34.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Como podemos observar el promedio de precio de los laptops es de 76745 pesos mexicanos y están entre 16990 hasta los 441990, también podemos ver que la RAM más pequeña es de 4Gb y la más grande de 32Gb. Los porcentajes que se encuentran de 25%, 50% y 75% hace referencia a los cuartiles, también se conoce el 50% como la media o promedio, podemos interpretar esto como diagrama boxplot que contiene los cuartiles antes vistos con sus datos atípicos y sus máximos y mínimos. La derivación estándar: esto significa como es la desviación estándar que tiene la media, una desviación estándar más alta indica una mayor dispersión de los datos alrededor de la media, mientras la desviación baja indica una menor dispersión. Esto también puede estar involucrado con el diagrama de boxplot en donde sería el máximo bigote y su mínimo, en el caso de la tarjeta gráfica podemos observar que tiene un indicador con menor dispersión de 0.882 en cambio el hdd si es diferente con 411.4.

Programación:



Figura 60: Programación 35.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Este código muestra los tipos de datos de cada columna en el conjunto llamándolos así datos ya por último la función "dtypes" ayuda a obtener los tipos de datos de cada columna. Mostrando así la información sobre laptops, la compañía, productos y entre otros como también nombra el tipo de variables que tiene cada una.

Ejecución:

```
brand      object
processor_brand  object
processor_name  object
processor_gnrtn  float64
ram_          int64
ram_type      object
ssd           int64
hdd           int64
os            object
os_bit        object
graphic_card_  int64
weight        object
warranty       float64
Touchscreen    object
msoffice       object
Price         int64
rating         int64
Number of Ratings  int64
Number of Reviews  int64
dtype: object
```

Figura 61: Ejecución 35.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Nuestro conjunto de datos tiene información de tipo object lo dejamos de esta forma ya que no es información que se usara entonces la dejaremos como cadena de texto, la información que se usara son laptop_ID, Inches, Price_euros y Company por esta razón estas si están en sus variables correspondientes.

Programación:

```
Modelos de regresion Lineal y no Lineal

[ ] mod0 = smf.ols('processor_gnrtn ~ Price',pima).fit()
mod0.params
```

Figura 62: Programación 36.

Fuente: Propia, echo en Google Colab.

Procedimientos:

En este análisis, se ajustó un modelo de regresión lineal simple para examinar la relación entre la variable dependiente "processor_gnrtn" y la variable independiente "Price". El modelo de regresión lineal simple supone una relación lineal entre la variable dependiente y la variable independiente, y se puede expresar como:

$$\text{processor_gnrtn} = \beta_0 + \beta_1 * \text{Price} + \varepsilon$$

Donde β_0 representa el valor de intersección (el valor esperado de processor_gnrtn cuando Price es cero), β_1 representa la pendiente (el cambio esperado en processor_gnrtn

por cada unidad de cambio en Price), y ϵ representa el término de error aleatorio.

Los coeficientes estimados del modelo se muestran en la salida: β_0 (Intercept) = 10.47158 y β_1 (Price) = -8.962790e-07. Estos valores sugieren que, manteniendo todo lo demás constante, un aumento de una unidad en Price se asocia con una disminución de 8.962790e-07 unidades en processor_gnrtn.

Es importante tener en cuenta que los coeficientes estimados son específicos para el conjunto de datos utilizado y pueden variar si se utilizan diferentes conjuntos de datos. Además, es necesario evaluar la significancia estadística y la bondad de ajuste del modelo antes de confiar en las interpretaciones de los coeficientes.

Programación:

```
[ ] import statsmodels.formula.api as smf
import matplotlib.pyplot as plt

# Obtener los datos
x = pima['processor_gnrtn']
y = pima['Price']

# Crear el modelo de regresión lineal
mod0 = smf.ols('y ~ x', data=pima).fit()

# Obtener los coeficientes del modelo
b0 = mod0.params[0]
b1 = mod0.params[1]

# Graficar los datos y la línea de regresión
plt.figure(figsize=(8, 6))
plt.scatter(x, y, s=50)
plt.plot(x, b0 + b1*x, color='red', linewidth=2)

# Agregar etiquetas y título
plt.xlabel('processor_gnrtn')
plt.ylabel('Price')
plt.title('Regresión Lineal')
```

Figura 63: Programación 36.

Fuente: Propia, echo en Google Colab.

```
plt.show()

# Mostrar la ecuación de la regresión
equation = f'Price = {b0:.2f} + {b1:.2f} * x'
print("La ecuación de la regresión es:", equation)

# Obtener el valor de r^2
r_squared = mod0.rsquared
print("El valor de r^2 para la regresión lineal es:", r_squared)
```

Figura 64: Programación 37.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Este código realiza una regresión lineal simple entre las variables 'processor_gnrtn' (eje x) y 'Price' (eje y) del conjunto de datos Pima. Primero, crea un modelo de regresión lineal utilizando el método de mínimos cuadrados ordinarios (OLS) de statsmodels. Luego, ajusta el modelo a los datos y extrae los coeficientes de la línea de regresión. Después, genera un gráfico que muestra los puntos de datos y la línea de regresión. Añade etiquetas a los ejes y un título al gráfico. Finalmente, muestra la ecuación de la línea de regresión y el valor de r^2 , que representa la proporción de la varianza en la variable dependiente que es predecible a partir de la variable independiente.

Ejecución:

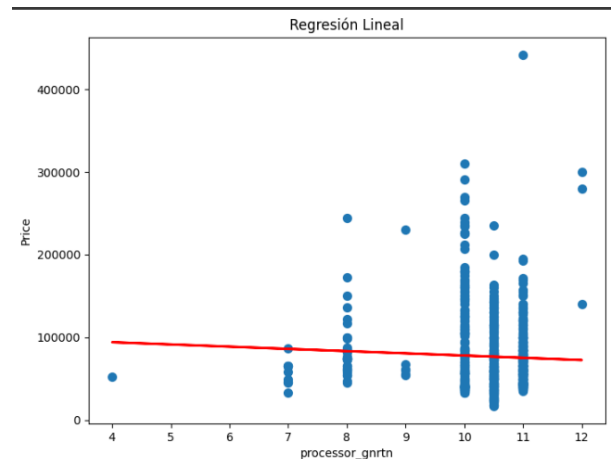


Figura 64: Ejecución 37.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Viendo los resultados obtenidos, la línea de tendencia en esta regresión polinómica (cubica) se puede definir como $\text{Price} = 559.5 x^3 - 1.639e+04 x^2 + 1.544e+05 x - 3.86e+05$, la cual tiene un 0.12 de coeficiencia ósea un 12% de relación, lo cual es relativamente baja, pero la más alta de las anteriores ya realizadas, lo cual se concluye que LA REGRESION POLINOMICA CUBICA es la regresión que mejor se adapta a la relación de processor_gnrtn y los Precios de los laptops.

Programación:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from scipy.optimize import curve_fit

# Obtener los datos
x = np.array(pima['processor_gnrtn'])
y = np.array(pima['Price'])

# Crear el modelo de regresión exponencial
def exponential_func(x, a, b):
    return a * np.exp(b * x)

# Ajustar el modelo a los datos originales
popt, pcov = curve_fit(exponential_func, x, y)

# Parámetros del ajuste
a, b = popt

# Graficar los datos originales y la curva ajustada
plt.scatter(x, y, label='Datos')
plt.plot(x, exponential_func(x, a, b), 'r--', label='Regresión Exponencial')
plt.xlabel('processor_gnrtn')
plt.ylabel('Price')
plt.legend()
plt.title('Regresión Exponencial')
plt.show()

# Calcular el valor de r^2
r2 = r2_score(y, exponential_func(x, a, b))
print("El valor de r^2 al cuadrado para la regresión exponencial es:", r2)

# Imprimir la ecuación de la regresión exponencial
print(f"Ecuación de la regresión exponencial: Price = {a:.2f} * e^{(b:.2f) * x}")
```

Figura 65: Programación 38.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Este código realiza una regresión exponencial sobre los datos de processor_gnrtn (eje x) y Price (eje y) del conjunto de datos pima. Primero, define una función exponencial llamada exponential_func con parámetros a y b, que representan los coeficientes de la regresión exponencial. Luego, ajusta el modelo exponencial a los datos originales utilizando el método curve_fit de la biblioteca scipy.optimize. Después de obtener los parámetros del ajuste, grafica los datos originales y la curva de regresión exponencial. Calcula el valor de r^2 para evaluar la bondad del ajuste. Finalmente, imprime la ecuación de la regresión exponencial obtenida.

Ejecución:

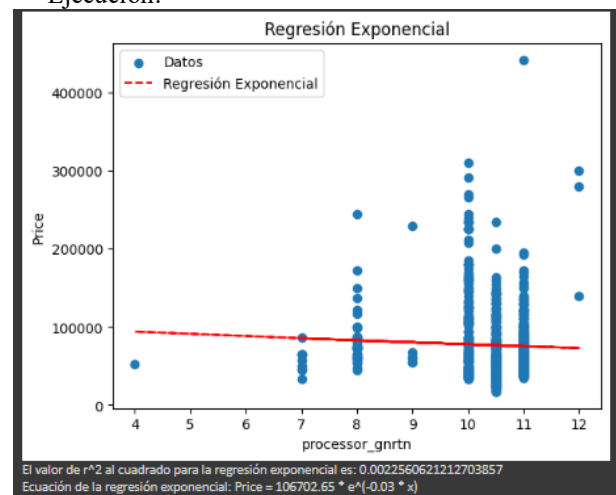


Figura 66: Ejecución 38.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Viendo los resultados obtenidos, la línea de tendencia en esta regresión exponencial se puede definir como $\text{Price} = 106702.65 * e^{(-0.03 * x)}$, la cual tiene un 0.002 de coeficiencia ósea un 0.2% de relación, lo cual es extremadamente baja y se relaciona con el bajo nivel de la regresión lineal, esto nos muestra que para obtener una mejor relación, se debe realizar otro tipo de relación que sea NO lineal ni exponencial.

NOTA: A pesar que la gráfica pareciera una ecuación lineal, se puede comprobar con el código que es una ecuación exponencial, creemos que esto se debe a que el intervalo donde se grafica la ecuación es justo la línea de descenso de la ecuación exponencial (Revise en GeoGebra que va de ascendente a descendente), se intentó realizar de diferentes métodos y en todos obteníamos este mismo resultado por lo que NO es un error del proceso, sino de el intervalo de ambos datos comparados en donde se muestra la gráfica.

Programación:

```
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

x=np.array(pima['processor_gnrtn'])
y=np.array(pima['Price'])

mymodel = np.poly1d(np.polyfit(x, y,3))

myline = np.linspace(min(x), max(x), len(x))
plt.scatter(x, y)
plt.plot(myline, mymodel(myline),color='red')
plt.xlabel('processor_gnrtn')
plt.ylabel('Price')
plt.title('Regresión polinómica (cúbica)')
plt.show()
y_predic = mymodel(myline)
from sklearn.metrics import r2_score
print("El valor de r^2: ", r2_score(y,y_predic))
print("Ecuación de la regresión polinómica:")
print("Price =")
print( mymodel)
```

Figura 67: Programación 39.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Este código realiza una regresión polinómica cúbica sobre los datos de processor_gnrtn (eje x) y Price (eje y) del conjunto de datos pima. Primero, ajusta un modelo polinómico de grado 3 a los datos originales utilizando el método polyfit de NumPy. Luego, crea un objeto mymodel de tipo poly1d que representa el modelo de regresión polinómica. A continuación, genera una línea de valores myline para graficar la curva de regresión. Grafica los datos originales y la curva de regresión polinómica cúbica. Calcula el valor de r^2 para evaluar la bondad del ajuste. Finalmente, imprime la ecuación de la regresión polinómica obtenida.

Ejecución:

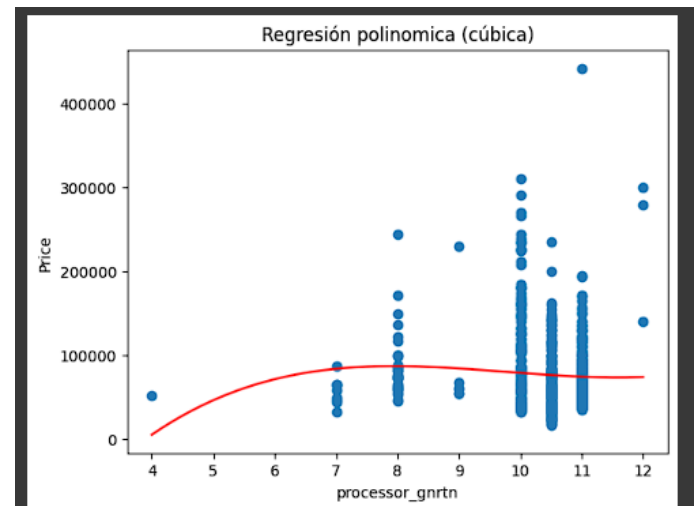


Figura 68: Ejecución 39.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Viendo los resultados obtenidos, la línea de tendencia en esta regresión polinómica (cubica) se puede definir como $\text{Price} = 559.5 x^3 - 1.639e+04 x^2 + 1.544e+05 x - 3.86e+05$, la cual tiene un 0.12 de coeficiencia ósea un 12% de relación, lo cual es relativamente baja, pero la más alta de las anteriores ya realizadas, lo cual se concluye que LA REGRESION POLINOMICA CUBICA es la regresión que mejor se adapta a la relación de processor_gnrtn y los Precios de los laptops.

Modelos de relación opcionales.

Para complementar nuestras hipótesis, sugerimos aplicar varios modelos de relación y métodos de visualización de datos. Utilizamos histogramas para analizar la distribución de variables clave, como las calificaciones (ratings) y las tendencias de compra, lo que nos permite identificar patrones y distribuciones de frecuencia. Además, empleamos gráficos de dispersión (diagramas de dispersión) para explorar la relación entre la generación del procesador (processor_gnrtn) y el precio de las computadoras, ayudándonos a visualizar posibles correlaciones y tendencias. También utilizamos diagramas de caja (boxplots) para examinar la dispersión y la variabilidad de los precios en diferentes generaciones de procesadores, así como para identificar posibles valores atípicos. Programación:

```
[ ] sns.displot(pima['processor_gnrtn'])
plt.title('Frecuencia generación de procesador')
```

Figura 69: Programación 40.

Fuente: Propia, echo en Google Colab.

Procedimiento:

El comando `sns.histplot(pima['rating'].dropna(), kde=True)` en Python, utilizando Seaborn, crea un gráfico que representa la distribución de los datos en la columna 'rating' del DataFrame pima, después de eliminar valores nulos. Este gráfico incluye un histograma que muestra la frecuencia de los valores y una estimación de la densidad de kernel (KDE) para suavizar la distribución, lo que facilita la comprensión de la forma y la dispersión de los datos en la columna 'rating'.

Ejecución:

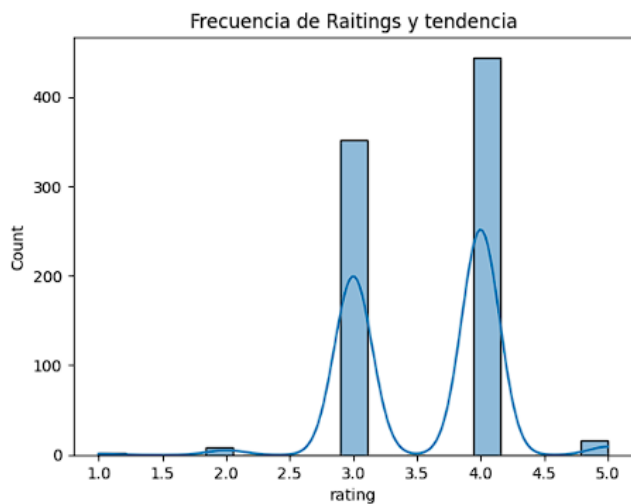


Figura 70: Ejecución 40.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Este histograma de Seaborn muestra la cantidad de raiting según su número de calificación, enseñando que la mayoría de las laptops tienen calificación de 4.0 seguido de 3.0, además muestra una línea de tendencia que complementa la gráfica e intuye donde se encuentran la mayoría de las laptops con respecto al raiting.

Programación:

```
sns.histplot(pima['processor_gnrtn'].dropna(),kde=True)
plt.title('Frecuencia generaciones de procesador y tendencia')
```

Figura 71: Programación 41.

Fuente: Propia, echo en Google Colab.

Procedimiento.

El comando `sns.histplot(pima['rating'].dropna(), kde=True)` en Python, utilizando Seaborn, crea un gráfico que representa la distribución de los datos en la columna 'rating' del DataFrame pima, después de eliminar valores nulos. Este gráfico incluye un histograma que muestra la frecuencia de los valores y una estimación de la densidad de kernel (KDE) para suavizar la distribución, lo que facilita la comprensión de la forma y la dispersión de los datos en la columna 'rating'.

Ejecución:

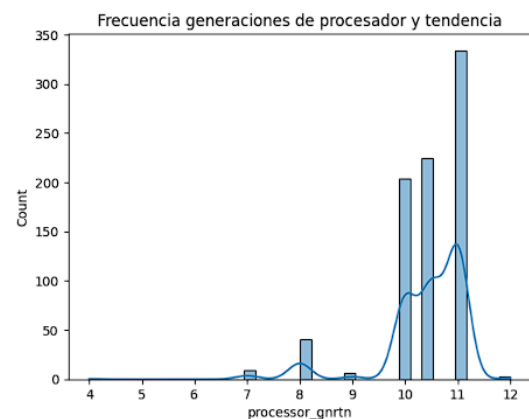


Figura 72: Ejecución 41.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Este histograma de Seaborn muestra la cantidad de Laptops según la generación de procesador que manejan, enseñando que la mayoría de las laptops tienen un procesador de 11ava generación seguido de 10ava y 10,5ava, además muestra una línea de tendencia que complementa la gráfica y intuye donde se encuentran la mayoría de las laptops con respecto a generación del procesador.

Programación:

```
[ ] import pandas as pd
import seaborn as sns

import seaborn as sns
import matplotlib.pyplot as plt

pimad = pima['rating'].dropna().sort_values()
pimadp = pima['Price'].dropna().sort_values()

plt.figure(figsize=(12, 6))
sns.lineplot(x=pimad, y=pimadp)
plt.xlabel('rating')
plt.ylabel('Prices')
plt.title('Rating vs Precio')
plt.show()
```

Figura 73: Programación 42.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Procedimientos:

Este código utiliza las bibliotecas Pandas, Seaborn y Matplotlib para trazar un gráfico de línea que muestra la relación entre las calificaciones ('rating') y los precios ('Price') de un conjunto de datos llamado 'pima'. Primero, se eliminan los valores nulos y se ordenan los valores de 'rating' y 'Price'. Luego, se crea un gráfico utilizando Seaborn, donde 'rating' se representa en el eje x y 'Price' en el eje y, con etiquetas adecuadas para los ejes y un título. Finalmente, el gráfico se muestra usando Matplotlib.

Ejecución:

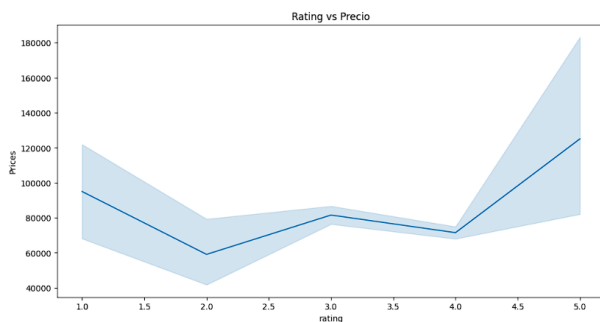


Figura 74: Ejecución 42.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Conclusiones:

En esta grafica vemos los picos de la generación de los Raiting con su precio y los picos más altos están entre 4 y 5 ya que se encuentran los precios más comunes, y tenemos uno en

2 el cual presenta el precio más bajo, incluso más bajo que computadores con rating.

Los laptops de 120.000 pesos mexicanos tienen la mejor calificación que los de la competencia, los laptops de una calificación de 3 que sus precios son de 80.000 pesos mexicanos brindan una buena experiencia al consumidor pero no la mejor, sin embargo es la opción que más se puede llegar a ajustar al presupuesto del cliente.

Programación:

```
[ ] import pandas as pd
import seaborn as sns

import seaborn as sns
import matplotlib.pyplot as plt

pimad = pima['processor_gnrtn'].dropna().sort_values()
pimadp = pima['Price'].dropna().sort_values()

plt.figure(figsize=(12, 6))
sns.lineplot(x=pimad, y=pimadp)
plt.xlabel('processor_gnrtn')
plt.ylabel('Prices')
plt.title('Generacion del Procesador vs Precio')
plt.show()
```

Figura 75: Programación 43.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Procedimientos:

Selecciona la columna 'processor_gnrtn' del DataFrame pima, elimina los valores nulos y ordena los valores de manera ascendente. Selecciona la columna 'Price' del DataFrame pima, elimina los valores nulos y ordena los valores de manera ascendente. Creamos una figura con un tamaño de 12 pulgadas de ancho y 6 pulgadas de alto. Crea un gráfico de líneas utilizando Seaborn, donde los valores de procesador se utilizan en el eje x y los valores de Precios se utilizan en el eje y. Para terminar, escribimos el título y mostramos la gráfica.

Ejecución:

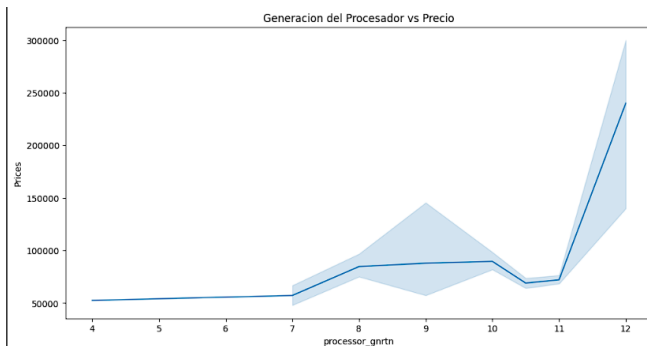


Figura 76: Ejecución 43.

Fuente: Propia, echo en Google Colab.

Conclusiones:

En esta grafica vemos los picos de la generación de los procesadores con su precio y los picos más altos están entre la generación 8 a la 10 ya que se encuentran los precios más comunes, y tenemos uno en el 12 ya que tenemos una compra con un precio muy alto como se e ven en las otra graficas. Los procesadores de 8 a 10 generación son los más frecuentes debido a que las multinacionales venden productos de bajo costo y con un rendimiento decente para tratar de satisfacer la demanda de que se pide, por lo general los últimos procesadores traen más núcleos y eso hace que aumente drásticamente el rendimiento y la velocidad de una laptop.

Programación.

```
sns.scatterplot(x='Price',y='rating',data=pima,s=100)
plt.title('Precios vs Raiting')
```

Figura 77: Programación 44.

Fuente: Propia, echo en Google Colab.

Procedimientos:

`sns.scatterplot(x='Price',y='rating',data=pima,s=100)` Crea un gráfico de dispersión donde el eje x representa los precios (Price) y el eje y representa la generación del calificación (rating). Se utiliza el DataFrame pima como fuente de datos. El parámetro `s=100` establece el tamaño de los puntos en el gráfico. Establecemos el título del gráfico como 'Precios vs calificación' y mostramos el gráfico.

Ejecución:

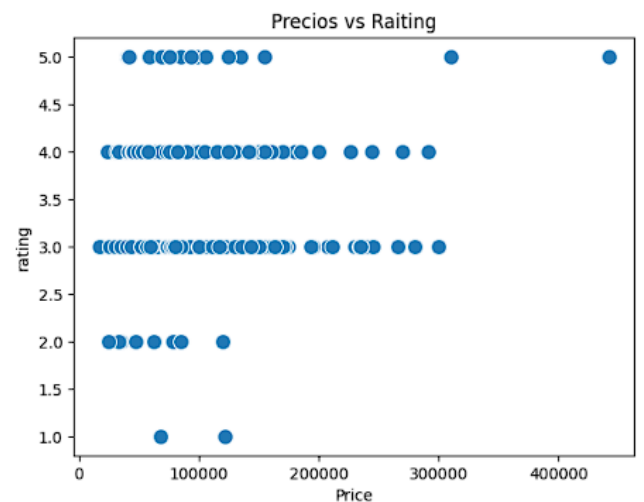


Figura 78: Ejecución 43.

Fuente: Propia, echo en Google Colab.

Conclusiones

Las más calificadas son el 3, 4 y 5 entre estos los precios más calificados son de 0 a 200,000, entonces según al grafica es recomendable comprar computadores que rondan entre los 0 a 200,000 aun así estos precios son accesibles ya que no son una cantidad tan grande y tiene muy buenas calificaciones aun así esto depende de la marca. El raiting en esta grafica se puede interpretar, como una buena nota ya que supera la mitad de la calificación para laptops.

Programación:

```
sns.scatterplot(x='Price',y='processor_gnrtn',data=pima,s=100)
plt.title('Precios vs Generación del procesador')
```

Figura 79: Programación 44.

Fuente: Propia, echo en Google Colab.

Procedimiento:

`sns.scatterplot(x='Price', y='processor_gnrtn', data=pima, s=100)`: Crea un gráfico de dispersión donde el eje x representa los precios (Price) y el eje y representa la generación del procesador (processor_gnrtn). Se utiliza el DataFrame pima como fuente de datos. El parámetro `s=100` establece el tamaño de los puntos en el gráfico. Establecemos el título del gráfico como 'Precios vs Generación del procesador' y mostramos el gráfico.

Ejecución:

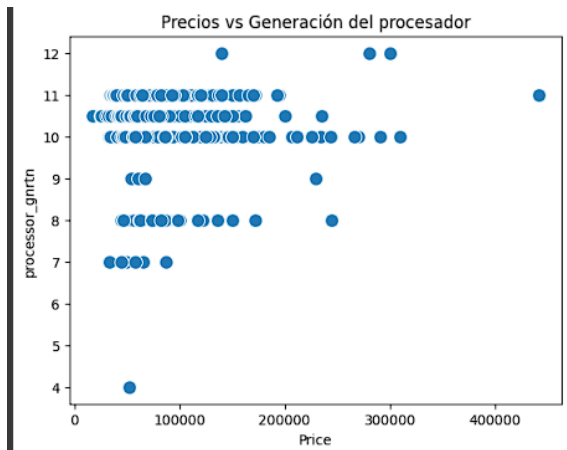


Figura 80: Ejecución 44.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La generación que fue más comprada es la 10 y 11 teniendo gran variedad de precios aun así sus fuertes están entre 0 a 200000 y en la generación 11 se encuentra el precio más alto siendo más de 4000000 como también se puede ver la menos comprada es la 4.

Programación:

```
sns.boxplot(x='rating', y='Price', data=pima)
plt.title('Box Plot de Precio por Raiting')
```

Figura 81: Programación 45.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Creamos un gráfico de caja donde el eje x representa el raiting y el eje y los precios utilizando los datos de nuestro DataFrame. Para la siguiente parte solo quedaría agregar el titulo y mostrar la gráfica.

Ejecución:

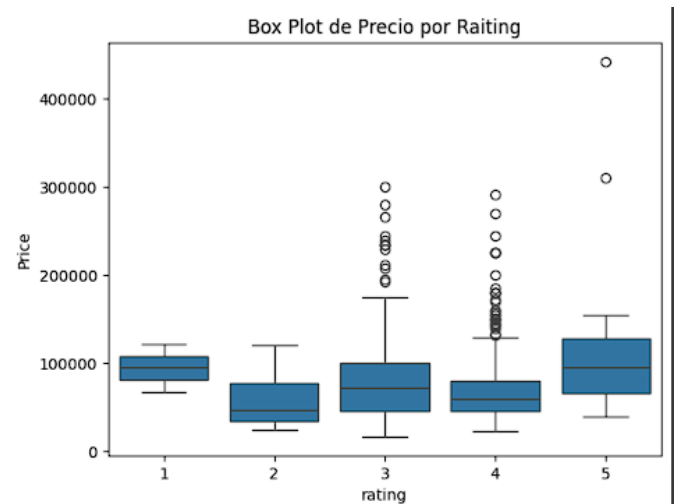


Figura 82: Ejecución 45.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Nuestras conclusiones empiezan desde los 3 hasta los 5, la más calificada está en la posición 4 y ronda de los 100000 hasta los 300000 y donde tiene mayor incremento es entre los 150000 hasta los 200000, en el puto 5 solo hay 2 donde sus precios son altos puede ser por esto que están en esta posición

Programación:

```
sns.boxplot(x='processor_brand', y='Price', data=pima)
plt.title('Box Plot de Precio por Marca de procesador')
plt.show()
```

Figura 83: Programación 46.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Creamos un gráfico de caja donde el eje x representa las marcas del procesador y el eje y los precios utilizando los datos de nuestro DataFrame. Para la siguiente parte solo quedaría agregar el titulo y mostrar la gráfica.

Ejecución:

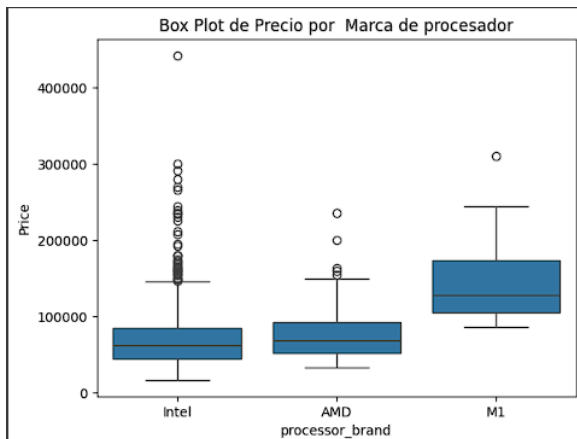


Figura 83: Ejecución 46.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Como se puede mostrar la marca más comprada es la Intel donde su valor más comprado están entre los 100000 y 300000, en la marca AMD tenemos muy pocas ventas comparadas a Intel estos precios rondan entre 160000 y 250000 para terminar el M1 solo cuenta con una venta de 300000.

Programación:

```
#Price Histogram
prices = pima ['Price']

# Create a histogram for the 'prices' variable
plt.figure(figsize=(8, 4))
sns.histplot(prices, bins=40, color='blue', kde=True) # Using seaborn for his
plt.title('Histograma de Prices con KDE')
plt.xlabel('Prices')
plt.ylabel('Frecuencia')

# Add a vertical line for the mean of prices
plt.axvline(prices.mean(), color='red', linestyle='dashed', linewidth=2, label

# Add tick marks at every 500 on the x-axis
plt.xticks(range(0, int(prices.max()) + 100000, 100000))

plt.legend() # Display legend for the line
plt.show()

# Print the mean price
print('El Precio principal de los laptop es: {:.2f}'.format(prices.mean()))
```

Figura 84: Programación 47.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Para comenzaremos llamado a nuestra columna de “Price” de nuestro data frame y se le asignara la variable de “prices”. Ya con esto vamos a crear una figura de un histograma con 8 pulgadas de anchura y 4 de altura, después de esto se utiliza

“Seaborn” para establecer el número de contenedores en 40, el color en azul y se incluye una estimación de la densidad del núcleo (KDE). Para que nuestro diagrama se vea más presentable ponemos un título como también pondremos etiquetas a los ejes x y y, Agrega una línea vertical en el valor medio de los precios (prices.mean()), de color rojo, con un estilo de línea discontinua y un ancho de línea de 2. Se agrega una etiqueta 'Precio principal' para esta línea. Ya para terminar estableceremos una marca de los ejes x en incrementos de 100000 desde 0 hasta el valor máximo, solo nos quedaría mostrar nuestra gráfica y la línea que representa el precio medio.

Ejecución:

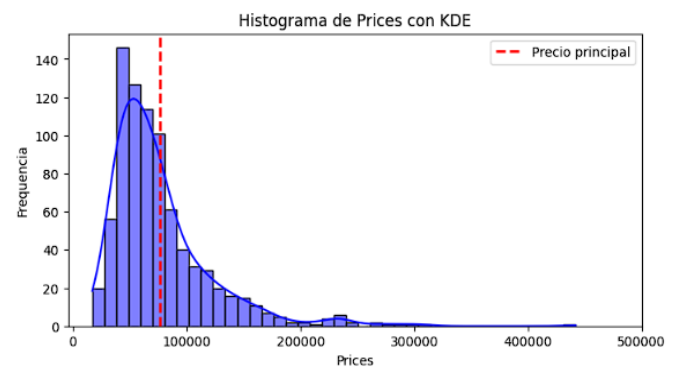


Figura 85: Ejecución 47.

Fuente: Propia, echo en Google Colab.

Conclusiones:

En la gráfica podemos observar que nuestro valor medio o principal es de 76745.18 la gráfica lo tiene muy al inicio ya que gran parte de nuestros valores se encuentran ahí, ya después de los 100000 esta grafica empieza a decaer y nuestro KDE que es una estimación de la dimensión de nuestro histograma de adapta muy bien a nuestra grafica.

Programación:

```
sns.scatterplot(x='processor_gnrtn',y='Price',data=pima,
style = "warranty",alpha=0.3)
plt.title('Generación del procesador vs Raiting con garanti
```

Figura 86: Programación 48.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Para empezar, usaremos la función “scatterplot” de una biblioteca ya antes importada para crear un gráfico de dispersión entre dos variables “processor_gnrtn” en el eje x y “Price” en el eje y. Primero crearemos la gráfica con la función ya antes explicada como también sus ejes, después utilizamos nuestro DataFrame “prima”. Después de esto usamos “style=“warranty”” esta opción asignará un estilo diferente a cada categoría única presente en la columna “warranty” del DataFrame, por esta razón en el grafico hay diferentes formas como puntos o el signo suma, después se ajustará la transparencia de los puntos con “alpha=0.3”.

Ejecución:

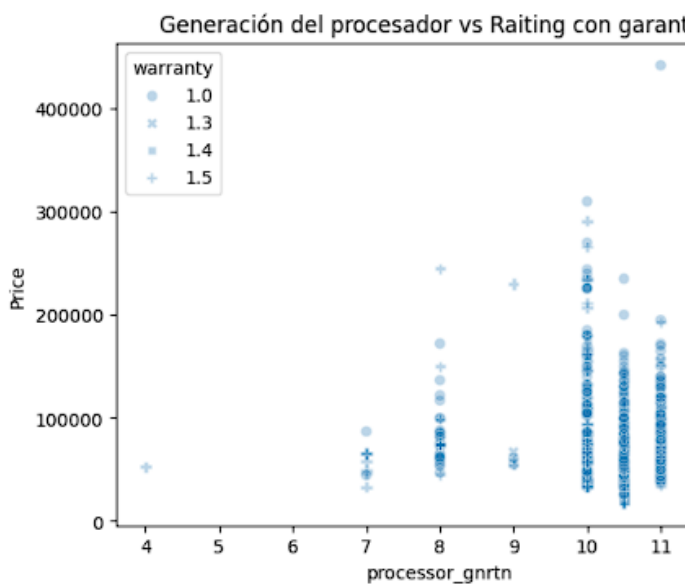


Figura 87: Ejecución 48.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Para esta grafica visualizaremos todas las generaciones en una misma gráfica y se puede observar que los procesadores más comprados están entre el 10 y 11 y los menos comprados son el 4 y 12 como también los precios más comunes están entre los 100000 y 200000, además las mayores garantías se encuentran en los procesadores más nuevos.

Programación:

```
[ ] sns.relplot(x='processor_gnrtn',y='Price',data=pima,
               col = "warranty")
```

Figura 87: Programación 49.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Para comenzar llamaremos la función “relplot” de una librería ya antes importada “seaborn” esta nos será útil para crear gráficos de relación, tales como gráficos de dispersión. En nuestra grafica necesitaremos nombrar el eje x como también nuestro eje y. Nuestro eje x va a ser “processor_gnrtn” o generación del procesador y nuestro eje y será “Price” o precio, estos datos se encuentran en nuestro DataFrame llamado “pima”. Para finalizar utilizamos “col=“warranty”” para dividir nuestro grafico en diferentes paneles basados en los valores únicos de nuestra columna garantía.

Ejecución:

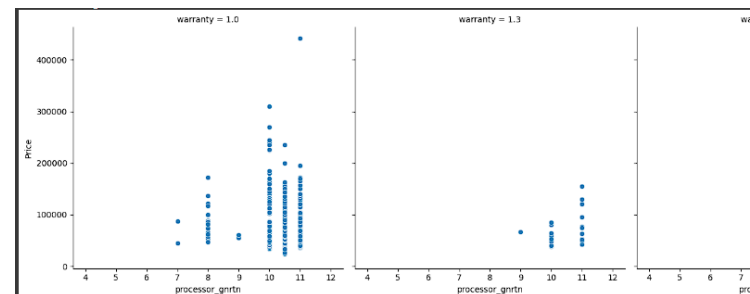


Figura 88: Ejecución 49.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Como podemos observar en la gráfica encontramos que en la primera generación 1.0 los procesadores más comprados son los 10, 11 y 12 como también, su precio no baria tanto de los otros. En la generación 1.3 hay menos compras comparadas con la anterior y sus precios son bajos y los procesadores de una generación alta. En la generación 1.4 podemos observar que los precios siguen siendo bajos y el procesador más comprado es el 10. Para finalizar en la generación 1.5 hay bastante variación como de procesadores desde 4 a al 12 pero el más frecuente es el 10 con precios bajos como altos pero los precios bajos.

Resultados finales:

Este estudio examina diferentes modelos de regresión para determinar la relación entre la generación de procesadores (processor_gnrtn) y los precios de laptops. Se evaluaron regresiones lineales, exponenciales y polinómicas cúbicas. El

análisis reveló que la regresión polinómica cúbica es el modelo que mejor se adapta a los datos, aunque su coeficiente de determinación (R^2) es relativamente bajo. Además, se realizaron análisis adicionales sobre la distribución de calificaciones de laptops y su relación con los precios y generaciones de procesadores, proporcionando una visión más completa del mercado.

Conclusión final:

La regresión lineal, con una ecuación de $\text{Price} = 104799.12 - 2696.77 * x$ y un coeficiente de determinación (R^2) de 0.002 (0.2%), mostró una relación extremadamente baja entre la generación de procesadores y los precios de las laptops. Esto indica que este modelo no es adecuado para describir la relación entre estas variables. De manera similar, la regresión exponencial, con una ecuación de $\text{Price} = 106702.65 * e^{(-0.03 * x)}$ y un R^2 también de 0.002 (0.2%), no proporciona una mejor descripción de la relación. Ambos modelos lineal y exponencial fallan en capturar la complejidad de la relación entre la generación de procesadores y los precios de las laptops.

Por otro lado, la regresión polinómica cúbica presentó una ecuación de $\text{Price} = 559.5 * x^3 - 1.639e+04 * x^2 + 1.544e+05 * x - 3.86e+05$ y un R^2 de 0.12 (12%). Aunque este coeficiente de determinación es relativamente bajo, indica que la relación es más compleja y se ajusta mejor con un modelo polinómico cúbico. Esto sugiere que existen factores no lineales que afectan los precios de las laptops en función de la generación de procesadores.

Los análisis adicionales mostraron que la mayoría de las laptops tienen calificaciones de 3.0 a 4.0. Las generaciones de procesadores más compradas son la 10ª y 11ª, mientras que la 4ª y 12ª son las menos comunes. Los precios más frecuentes se encuentran entre 100,000 y 200,000 pesos mexicanos. Estos datos indican que los consumidores tienden a preferir laptops con procesadores de generaciones recientes (10ª y 11ª) y que las laptops con calificaciones intermedias (3.0 a 4.0) son las más comunes.

En términos de precios y calificaciones, se observó que los laptops con mejores calificaciones tienden a estar en un rango de precios de 120,000 pesos mexicanos, mientras que los laptops con calificaciones de 3 tienen precios de alrededor de 80,000 pesos mexicanos. Esto sugiere que, aunque los laptops de menor calificación son más económicos, los consumidores que buscan una mejor experiencia de usuario están dispuestos a pagar más por laptops de mayor calificación.

La relación entre la generación de procesadores y los precios también mostró que los procesadores de 8ª a 10ª generación son los más frecuentes, probablemente debido a que ofrecen un equilibrio entre costo y rendimiento. Los procesadores de

última generación, aunque ofrecen un rendimiento superior debido a más núcleos y mayor velocidad, tienen un costo significativamente más alto.

En conclusión, la regresión polinómica cúbica se identificó como el mejor modelo para describir la relación entre la generación de procesadores y los precios de laptops, aunque con un R^2 relativamente bajo. Esto indica que hay una relación compleja entre estas variables que los modelos lineales y exponenciales no pueden capturar adecuadamente. Los análisis adicionales de calificaciones y distribuciones de precios proporcionan una visión complementaria del mercado, destacando las preferencias de los consumidores y las estrategias de precios. Estos hallazgos son útiles para fabricantes y vendedores de laptops, ya que pueden informar estrategias de precios y marketing, sugiriendo que se consideren modelos más complejos para un mejor entendimiento de las relaciones entre características de los productos y sus precios.

Corte 3:

En esta tercera y última entrega nos enfocaremos en las teorías de la probabilidad estadística que mejor se adapten al proyecto, definiremos una teoría como la que mejor se adapte a las variables “Precio en Euros” para el primer análisis y para el segundo la variable “Tamaño en pulgadas”, consideramos que es fundamental concluir este análisis con estas teorías, pues explicadas en palabras de José Francisco López en el medio Economipedia, se menciona la importancia de estas teorías; “La teoría de las probabilidades estadísticas es una rama de las matemáticas que se enfoca en el análisis y la interpretación de fenómenos aleatorios. Utiliza conceptos y herramientas de probabilidad para describir y predecir el comportamiento de sistemas complejos donde intervienen variables aleatorias”, en nuestro caso, estas variables que elegimos son excelentes por su variabilidad y la importancia de hallar un valor esperado como la varianza de cada variable, concluiremos este análisis comparando resultados del primer corte y brindando conclusiones finales que notamos durante el desarrollo según nuestro criterio Ingenieril

Programación:

```
[ ] datos.describe()
```

Figura 89: Programación 50.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Como se puede observar lo primero que hacemos es importar la biblioteca “drive” y con

“drive.mount('/content/drive')” nos permite acceder a los archivos almacenados de Google Drive pero desde Colab. Ya con esto, nombramos la variable de datos que va a tomar el DataFrame, nos ayudara para poder mostrar las primeras 1002 filas del DataFrame. Finalmente generamos “data.describe()” que como hemos visto en clase nos ayudara para poder hallar estadísticas sobre el DataFrame.

Ejecución:

```
laptop_ID      int64
Company        object
Product        object
TypeName       object
Inches         float64
ScreenResolution object
Cpu            object
Ram            object
Memory         object
Gpu            object
OpSys          object
Weight         object
Price_euros    float64
dtype: object
```

Figura 90: Ejecución 50.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Como podemos observar el promedio del tamaño de la pantalla es de 15,02 pulgadas y están entre 14 a 16 pulgadas, también podemos ver que la pantalla más pequeña es de 10.1 pulgas y la más grande de 18.4. Con relación a los precios que están dados en euros, esta ronda entre los 599 euros y 1487 euros, con un minio de 174 euros y un máximo de 6099 euros. Los porcentajes que se encuentran de 25%, 50% y 75% hace referencia a los cuartiles, también se conoce el 50% como la media o promedio, podemos interpretar esto como diagrama boxplot que contiene los cuartiles antes vistos con sus datos atípicos y sus máximos y mínimos La derivación estándar: esto significa como es la desviación estándar que tiene la media, una desviación estándar más alta indica una mayor dispersión de los datos alrededor de la media, mientras la desviación baja indica una menor dispersión. Esto también puede estar involucrado con el diagrama de boxplot en donde sería el máximo bigote y su mínimo, en el caso de la pantalla podemos observar que tiene un indicador con menor dispersión de 1.4 en cambio el precio si es diferente con 699 euros.

Programación:

Tipos de datos en el DataSet:

```
[ ] datos.dtypes
```

Figura 91: Programación 51.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Este código muestra los tipos de datos de cada columna en el conjunto llamándolos así datos ya por último la función “dtypes” ayuda a obtener los tipos de datos de cada columna. Mostrando así la información sobre laptops, la compañía, productos y entre otros como también nombra el tipo de variables que tiene cada una

Ejecución:

```
laptop_ID      int64
Company        object
Product        object
TypeName       object
Inches         float64
ScreenResolution object
Cpu            object
Ram            object
Memory         object
Gpu            object
OpSys          object
Weight         object
Price_euros    float64
dtype: object
```

Figura 92: Ejecución 51.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Nuestro conjunto de datos tiene información de tipo object lo dejamos de esta forma ya que no es información que se usara entonces la dejaremos como cadena de texto, la información que se usara son laptop_ID, Inches, Price_euros y Company por esta razón estas si están en sus variables correspondientes.

Programación:

```
[ ] important_features = ['laptop_ID', 'Company', 'Product', 'TypeName', 'Inches', 'ScreenResolution']
datos_data = datos.loc[:, important_features]
datos_data.describe()
```

Figura 93: Programación 52.

Definir Intervalos (Bins): `np.linspace` genera intervalos equiespaciados basados en el mínimo y máximo de `Price_euros`. Calcular Frecuencia: `np.histogram` cuenta la cantidad de datos que caen en cada intervalo. Calcular Probabilidad: Se divide la frecuencia de cada intervalo por la suma total de frecuencias. Calcular Probabilidades Acumuladas: `np.cumsum` calcula la suma acumulativa de las probabilidades. Crear DataFrame: Se crea un DataFrame con los intervalos, la frecuencia, la probabilidad puntual ($P(X=x)$) y la probabilidad acumulada ($P(X \leq x)$). Mostrar la Tabla: Se imprime la tabla de distribución de probabilidad en el formato solicitado. Calcular Media y Varianza: Se calculan la media y la varianza de los datos de `Price_euros`. Mostrar Media y Varianza: Se imprimen la media y la varianza. Al final se genera una tabla de distribución de probabilidad para la variable `Price_euros` y mostrará las probabilidades puntuales y acumuladas de cada intervalo, junto con la media y la varianza al final.

Ejecución:

```
Distribución de probabilidad de Los precios en Euros
r                                     tp(r)
P(X in (174.0, 766.5]) = 0.3638      0.3638
P(X in (766.5, 1359.0]) = 0.3438    0.7076
P(X in (1359.0, 1951.5]) = 0.1773    0.8849
P(X in (1951.5, 2544.0]) = 0.0752    0.9601
P(X in (2544.0, 3136.5]) = 0.0284    0.9885
P(X in (3136.5, 3729.0]) = 0.0061    0.9946
P(X in (3729.0, 4321.5]) = 0.0023    0.9969
P(X in (4321.5, 4914.0]) = 0.0015    0.9985
P(X in (4914.0, 5506.5]) = 0.0008    0.9992
P(X in (5506.5, 6099.0]) = 0.0008    1.0000

Media: 1123.69
Varianza: 488613.64
```

Figura 98: Ejecución 53.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La mayoría de los precios se encuentran en los primeros dos intervalos. El 36.38% de los precios están en el rango de (174.0, 766.5] y el 34.38% están en el rango de (766.5, 1359.0]. Esto significa que aproximadamente el 70.76% de los precios están por debajo de 1359 euros. Solo el 11.51% de los precios se encuentran por encima de 1951.5 euros, lo que sugiere que los precios más altos son relativamente raros.

Probabilidades Acumuladas: La probabilidad acumulada alcanza el 88.49% en el tercer intervalo (1359.0, 1951.5], lo que indica que casi el 90% de los precios están por debajo de 1951.5 euros. A partir del cuarto intervalo, las probabilidades

acumuladas aumentan más lentamente, mostrando que los precios en los intervalos superiores son mucho menos comunes. La media de los precios es de 1123.69 euros, lo que indica que el precio promedio se encuentra en el primer tercio del rango de precios. La varianza es bastante alta (488613.64), lo que indica que hay una gran dispersión de los precios en el dataset. Esto significa que, aunque muchos precios están cerca de la media, hay precios que se desvían significativamente de este valor.

Distribución teórica que más se ajusta:

Dado que la variable aleatoria es continua y positiva, y la distribución de probabilidad parece tener una forma asimétrica hacia la derecha, una distribución teórica candidata sencilla podría ser la distribución exponencial. La distribución exponencial se utiliza comúnmente para modelar el tiempo entre eventos en un proceso de Poisson, pero también se puede aplicar a otras variables aleatorias continuas positivas con una tasa de decaimiento constante. Para calcular el valor esperado (media) de una variable aleatoria que sigue una distribución exponencial en Python, podemos utilizar la función `expon.mean()` de la biblioteca `scipy.stats`.

Programación:

```
] import numpy as np
from scipy.stats import expon

# Datos de precios en euros
precios_euros = datos_data['Price_euros']

# Ajustar la distribución exponencial a los datos
parametro_lambda = 1 / precios_euros.mean() # Estimación
dist_exp = expon(scale=1/parametro_lambda)

# Calcular el valor esperado (media) de la distribución exponencial
valor_esperado = dist_exp.mean()

print(f"Valor esperado (media) de la distribución exponencial: {valor_esperado}")
```

Figura 98: Programación 54.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Estimamos el parámetro `lambda` de la distribución exponencial utilizando la inversa de la media de los precios en euros. Creamos una instancia de la distribución exponencial `dist_exp` con el parámetro `scale=1/parametro_lambda`. Calculamos el valor esperado (media) de la distribución exponencial utilizando `dist_exp.mean()`. Finalmente, imprimimos el valor esperado con dos decimales.

Ejecución:

Valor esperado (media) de la distribución exponencial ajustada: 1123.69

Figura 99: Ejecución 54.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La media de los datos de precios en euros (1123.69) coincide con el valor esperado de la distribución exponencial ajustada. Esto sugiere que la distribución exponencial podría ser una aproximación razonable para modelar la distribución de los precios en euros. La distribución exponencial es una distribución continua y positiva, lo que se ajusta a la naturaleza de los datos de precios, que son valores positivos. La distribución exponencial tiene una forma asimétrica positiva, lo que también coincide con la observación de que la distribución de probabilidad de los precios en euros está sesgada hacia la derecha (asimetría positiva).

Planteamiento de Problemas:

1. Supongamos que ajustamos una distribución exponencial a los datos de precios en euros. Calcular la probabilidad de que un precio seleccionado al azar sea menor o igual a 1000 euros.
2. Supongamos que ajustamos una distribución exponencial a los datos de precios en euros. Calcular el precio mínimo que será superado por el 20% de los datos, utilizando la distribución exponencial ajustada.
3. Calcular la probabilidad de que un precio seleccionado al azar se encuentre entre 500 y 2000 euros, utilizando la distribución teórica ajustada.

Problema 1:

Programación del problema 1.

```
[ ] import numpy as np
from scipy.stats import expon

# Datos de precios en euros
precios_euros = datos_data['Price_euros']

# Ajustar la distribución exponencial a los datos
parametro_lambda = 1 / precios_euros.mean() # Estimación del parámetro lambda
dist_exp = expon(scale=1/parametro_lambda)

# Calcular la probabilidad de que un precio sea menor o igual a 1000 euros
precio_maximo = 1000
probabilidad = dist_exp.cdf(precio_maximo)

print(f"La probabilidad de que un precio seleccionado al azar sea menor o igual a 1000 euros es: {probabilidad}")
```

Figura 100: Programación 55.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Estimamos el parámetro lambda de la distribución exponencial utilizando la inversa de la media de los precios en euros. Creamos una instancia de la distribución exponencial `dist_exp` con el parámetro `scale=1/parametro_lambda`. Definimos el precio máximo (`precio_maximo = 1000`) para el cual queremos calcular la probabilidad. Calculamos la probabilidad de que un precio seleccionado al azar sea menor o igual a 1000 euros utilizando la función de distribución acumulada (CDF) de la distribución exponencial (`dist_exp.cdf(precio_maximo)`). Finalmente, imprimimos la probabilidad con cuatro decimales.

Ejecución problema 1:

La probabilidad de que un precio seleccionado al azar sea menor o igual a 1000 euros es: 0.5893

Figura 101: Ejecución 55.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Según la distribución exponencial ajustada a los datos de precios en euros, la probabilidad de que un precio seleccionado al azar sea menor o igual a 1000 euros es de aproximadamente 0.5893 o 58.93%.

Problema 2:

Programación del problema 2:

```
import numpy as np
from scipy.stats import expon

# Datos de precios en euros
precios_euros = datos_data['Price_euros']

# Ajustar la distribución exponencial a los datos
parametro_lambda = 1 / precios_euros.mean() # Estimación del parámetro lambda
dist_exp = expon(scale=1/parametro_lambda)

# Calcular el precio mínimo que será superado por el 20% de los datos
probabilidad = 0.2 # 20%
precio_minimo = dist_exp.isf(probabilidad)

print(f"El precio mínimo que será superado por el 20% de los datos es: {precio_minimo}")
```

Figura 102: Programación 56.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Estimamos el parámetro λ de la distribución exponencial utilizando la inversa de la media de los precios en euros. Creamos una instancia de la distribución exponencial `dist_exp` con el parámetro `scale=1/parametro_lambda`. Definimos la probabilidad como 0.2 (20%). Calculamos el precio mínimo que será superado por el 20% de los datos utilizando la función `dist_exp.isf(probabilidad)`, que calcula el valor inverso de la función de distribución acumulada para una probabilidad dada. Finalmente, imprimimos el precio mínimo con dos decimales.

Ejecución problema 2:

 El precio mínimo que será superado por el 20% de los datos es: 1808.50 e

Figura 103: Ejecución 56.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Conclusiones:

Según la distribución exponencial ajustada a los datos de precios en euros, el precio mínimo que será superado por el 20% de los datos es de aproximadamente 1808.50 euros.

Problema 3:

Programación problema 3:

```
import numpy as np
from scipy.stats import expon

# Datos de precios en euros
precios_euros = datos_data['Price_euros']

# Ajustar la distribución exponencial a los datos
parametro_lambda = 1 / precios_euros.mean() # Estimación del parámetro lambda
dist_exp = expon(scale=1/parametro_lambda)

# Calcular la probabilidad de que un precio esté entre 500 y 2000 euros
precio_inferior = 500
precio_superior = 2000
probabilidad = dist_exp.cdf(precio_superior) - dist_exp.cdf(precio_inferior)

print(f"La probabilidad de que un precio seleccionado al azar esté entre {precio_inferior} y {precio_superior} es: {probabilidad}")
```

Figura 104: Programación 57.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Procedimiento:

Estimamos el parámetro λ de la distribución exponencial utilizando la inversa de la media de los precios en euros. Creamos una instancia de la distribución exponencial `dist_exp` con el parámetro `scale=1/parametro_lambda`. Definimos los límites inferior y superior del intervalo de precios (`precio_inferior = 500` y `precio_superior = 2000`).

Calculamos la probabilidad de que un precio seleccionado al azar se encuentre entre 500 y 2000 euros restando la función de distribución acumulada evaluada en el límite inferior (`dist_exp.cdf(precio_inferior)`) de la CDF evaluada en el límite superior (`dist_exp.cdf(precio_superior)`). Finalmente, imprimimos la probabilidad con cuatro decimales.

Ejecución problema 3:

La probabilidad de que un precio seleccionado al azar esté entre 500 y 2000 euros es: 0.4722

Figura 105: Ejecución 57.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Conclusiones:

Esto significa que, según la distribución exponencial ajustada a los datos de precios en euros, la probabilidad de que un precio seleccionado al azar se encuentre entre 500 y 2000 euros es de aproximadamente 0.4722 o 47.22%.

Programación problema 3:

```
import pandas as pd
important_features = ['laptop_ID', 'Company', 'Product', 'TypeName', 'Inches', 'ScreenResolution', 'Cpu']
datos_data = datos.loc[:, important_features]
df = datos_data['Price_euros'].value_counts()
bins = [0, 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000]
labels = ['0 - 1000', '1000 - 2000', '2000 - 3000', '3000 - 4000', '4000 - 5000', '5000 - 6000', '6000 - 7000', '7000 - 8000', '8000 - 9000', '9000 - 10000']
df['Clases'] = pd.cut(datos_data['Price_euros'], bins, labels=labels)
plot = df['Clases'].hist(edgecolor='black', linewidth=1.2)
plot.set_title("Precios en Euros por clases [€]")
plt.xlabel('€')
plt.show()
```

Figura 106: Programación 58.

Fuente: [Propia](#), [echo](#) en [Google](#) [Colab](#).

Ejecución problema 3:

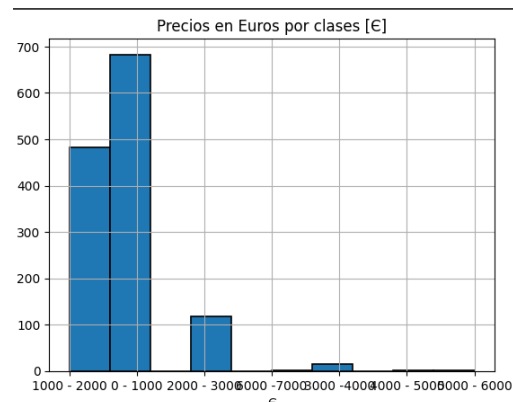


Figura 107: Ejecución 58.

Fuente: Propia, echo en Google Colab.

Histograma primer corte:

Recordemos el Histograma del precio en euros que realizamos en el primer corte, utilizaremos el mismo procedimiento. Recordando esto, Miremos ahora la gráfica de modelo de probabilidad que planteamos, el modelo exponencial, y compararemos sus resultados.

Programación problema 3:

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import expon

# Datos de precios en euros
precios_euros = datos_data['Price_euros']

# Ajustar la distribución exponencial a los datos
parametro_lambda = 1 / precios_euros.mean() # Estimación del parámetro lambda
dist_exp = expon(scale=1/parametro_lambda)

# Definir los valores de x para la gráfica
x_valores = np.linspace(precios_euros.min(), precios_euros.max(), 100)

# Calcular la función de densidad de probabilidad (PDF) para los valores de x
pdf_valores = dist_exp.pdf(x_valores)

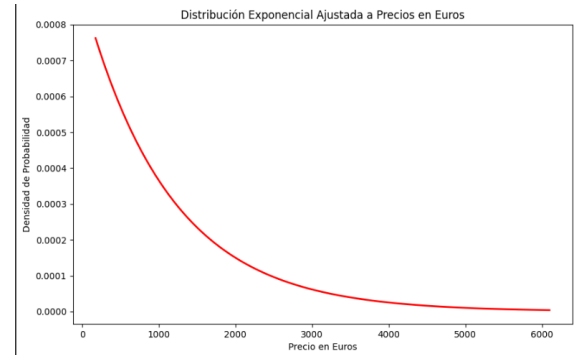
# Graficar la función de densidad de probabilidad
plt.figure(figsize=(10, 6))
plt.plot(x_valores, pdf_valores, 'r-', lw=2)
plt.xlabel("Precio en Euros")
plt.ylabel("Densidad de Probabilidad")
plt.title("Distribución Exponencial Ajustada a Precios en Euros")
plt.show()
```

Figura 108: Programación 59.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Estimamos el parámetro lambda de la distribución exponencial utilizando la inversa de la media de los precios en euros. Creamos una instancia de la distribución exponencial dist_exp con el parámetro scale=1/parametro_lambda. Definimos un rango de valores de x (x_valores) utilizando np.linspace desde el valor mínimo hasta el valor máximo de los precios en euros, con 100 puntos intermedios. Calculamos la función de densidad de probabilidad para los valores de x utilizando dist_exp.pdf(x_valores). Creamos una nueva figura con un tamaño personalizado (figsize=(10, 6)). Graficamos la función de densidad de probabilidad utilizando plt.plot(x_valores, pdf_valores, 'r-', lw=2). Agregamos etiquetas y un título a la gráfica. Finalmente, mostramos la gráfica utilizando plt.show().



Conclusiones:

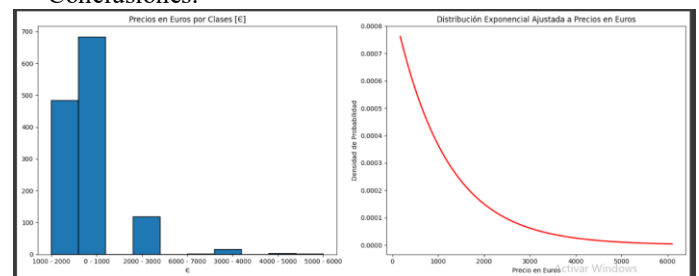


Figura 109: Ejecución 59.

Fuente: Propia, echo en Google Colab.

Como se evidencia, aunque no es 100% perfecto, el modelo de probabilidad exponencial se acopla a la variable precio en Euros, pues su semejanza va en el descenso de numero de computadores según el precio aumenta, siendo más común, los precios que van de los 1000 a 4000 Euros.

Segundo análisis:

Programación:

```
import pandas as pd
import numpy as np

# Definir el número de intervalos
num_bins = 4

# Crear los intervalos (bins)
bins = np.linspace(datos_data['Inches'].min(), datos_data['Inches'].max(), num_bins + 1)

# Calcular la frecuencia de datos en cada intervalo
counts, edges = np.histogram(datos_data['Inches'], bins=bins)

# Calcular la probabilidad para cada intervalo
probabilities = counts / counts.sum()

# Calcular las probabilidades acumuladas
cumulative_probabilities = np.cumsum(probabilities)

# Crear un DataFrame para la tabla de distribución de probabilidad
dist_prob_table = pd.DataFrame({
    'Interval': pd.IntervalIndex.from_breaks(edges),
    'Frequency': counts,
    'P(X=x)': probabilities,
    'P(X<=x)': cumulative_probabilities
})

print("\nDistribución de probabilidad de Los tamaños en pulgadas:")
print("r                tp(r)")
# Mostrar la tabla con el formato solicitado
for i, row in dist_prob_table.iterrows():
    print(f"P(X in {row['Interval']}) = {row['P(X=x)']:.4f} \t {row['P(X<=x)']:.4f}")

# Calcular la media y la varianza de los datos
mean_price = datos_data['Inches'].mean()
var_price = datos_data['Inches'].var()

# Mostrar la media y la varianza
print(f"\nMedia: {mean_price:.2f}")
print(f"Varianza: {var_price:.2f}")
```

Figura 110: Programación 60.

Fuente: Propia, echo en Google Colab.

Procedimiento:

Se realiza un análisis de la variable Inches en el conjunto de datos datos_data. Primero, importa las librerías necesarias (pandas y numpy) y define el número de intervalos (4 en este caso). Luego, utiliza np.linspace para crear los límites de los intervalos desde el valor mínimo hasta el máximo de Inches. Utiliza np.histogram para calcular la frecuencia de datos en cada intervalo y luego calcula las probabilidades dividiendo los conteos por el total de datos. Con np.cumsum, calcula las probabilidades acumuladas. Crea un DataFrame que contiene los intervalos, las frecuencias, las probabilidades y las probabilidades acumuladas. A continuación, itera sobre cada fila del DataFrame y la imprime en el formato solicitado. Finalmente, calcula la media y la varianza de Inches usando los métodos mean() y var() de pandas, y las imprime con dos decimales de precisión. Todo esto se hace para mostrar la distribución de probabilidad de los tamaños en pulgadas y proporcionar un resumen estadístico básico de los datos.

Ejecución:

```
Distribución de probabilidad de Los tamaños en pulgadas:
r                tp(r)
P(X in (10.1, 12.174999999999999]) = 0.0338    0.0338
P(X in (12.174999999999999, 14.25]) = 0.3223    0.3561
P(X in (14.25, 16.325]) = 0.5165    0.8726
P(X in (16.325, 18.4]) = 0.1274    1.0000

Media: 15.02
Varianza: 2.03
```

Figura 111: Ejecución 60.

Fuente: Propia, echo en Google Colab.

Conclusiones:

La mayoría de los tamaños se encuentran en los intervalos centrales, específicamente entre 12.17 y 16.33 pulgadas. Este rango cubre aproximadamente el 84% de los datos, lo que indica que los tamaños de productos están concentrados en este rango. El primer intervalo (10.1, 12.17] tiene una probabilidad bastante baja, sugiriendo que pocos productos tienen tamaños en este rango inferior. El último intervalo (16.33, 18.4] también tiene una probabilidad relativamente baja, indicando que pocos productos tienen tamaños en este rango superior. Media (15.02): La media de los tamaños es de 15.02 pulgadas, lo que sugiere que, en promedio, los productos tienden a estar cerca del centro de los intervalos mencionados. Varianza (2.03): La varianza de 2.03 pulgadas cuadradas indica una moderada dispersión de los tamaños alrededor de la media. Esto significa que aunque la mayoría de los tamaños están cerca de la media, hay una variabilidad suficiente para abarcar los intervalos definidos. El análisis sugiere que la mayoría de los tamaños de los productos se encuentran entre 12.17 y 16.33 pulgadas, con una media alrededor de 15.02 pulgadas y una varianza moderada que indica alguna variabilidad en los tamaños. Los extremos inferiores y superiores tienen menos productos, lo que puede influir en decisiones de producción y marketing, enfocándose en el rango central para satisfacer la demanda más común.

Distribución teórica que más se ajusta: La distribución de Poisson podría ser una opción razonable para modelar los datos de tamaños en pulgadas siempre que se trate de una variable aleatoria discreta que represente el número de eventos que ocurren en un intervalo de tiempo o espacio, los eventos ocurran de manera independiente y con una tasa constante, y la varianza de los datos sea relativamente pequeña en comparación con la media, lo cual sugiere que la media y la varianza podrían ser aproximadamente iguales, como lo asume la distribución de Poisson. Sin embargo, es importante verificar el cumplimiento de los supuestos y realizar pruebas de bondad de ajuste para confirmar si esta distribución se ajusta

adecuadamente a los datos en comparación con otras distribuciones discretas candidatas. CopyRetry.

Prueba Chi cuadrado (χ^2) para verificar si los datos se distribuyen como la Poisson

R.A. Fisher mostró que bajo el supuesto de que los datos $X_1, X_2, \dots, X_n$ son generados por una distribución de Poisson con algún parámetro λ (o bien μ), entonces el estadístico:

$$D = \sum_{i=1}^n (X_i - \bar{X})^2 / \bar{X}$$

donde $\bar{X}$ es la media muestral y tiene una χ^2 con $n-1$ grados de libertad. El p-valor de la cola superior está dado por:

$p=1-F(x)$ donde $F(x)$ es la distribución acumulada, la cual en Python se toma con la librería stats de scipy (python científico) como `chi2.cdf` (chi cuadrado de la cumulative distribution function), como se observa a continuación:

Programación:

```
# Código elaborado por Ing. Wilson Castro Z.
from scipy import stats
Dd = []
mu = np.mean(datos_data['Inches'])
for i in datos_data['Inches']:
    Dd.append((i-mu)**2)
D = sum(Dd)/mu
df = len(datos_data['Inches'])-1 # parametro de forma.
pvalor = 1-stats.chi2.cdf(D,df)
print(f'D = {D}\ndf = {df}\np-valor = {round(pvalor,3)}')
# Comparar el p-valor con un nivel de significancia (por ejemplo, 0.05)
alfa = 0.05
if pvalor < alfa:
    print("Se rechaza la hipótesis nula. Los datos no se ajustan a la distribución de Poisson.")
else:
    print("No se rechaza la hipótesis nula. Los datos se ajustan a la distribución de Poisson.")
```

Figura 112: Programación 61.

Fuente: Propia, echo en Google Colab.

Ejecución:

```
D = 176.3788515592255
df = 1302
p-valor = 1.0
No se rechaza la hipótesis nula. Los datos se ajustan a la distribución de Poisson.
```

Figura 113: Ejecución 61.

Fuente: Propia, echo en Google Colab.

Conclusiones:

como el $p\text{-value}=1.0 \geq 0.05$ -Tomando este último valor como el nivel de significancia ($\alpha=0.05$), concluimos que no hay evidencia en contra de la hipótesis nula de que la verdadera

distribución es de Poisson con una media de alrededor de 15.02 Pulgadas.

Teniendo en cuenta esto, realizamos la Distribución de Poisson para el tamaño en pulgadas:

Programación:

```
import numpy as np
from scipy.stats import poisson

# Datos de tamaños en pulgadas
tamanos_pulgadas = datos_data['Inches']

# Estimar el parámetro lambda de la distribución de Poisson
lambda_estimado = tamanos_pulgadas.mean()

# Crear una instancia de la distribución de Poisson
dist_poisson = poisson(mu=lambda_estimado)

# Calcular el valor esperado de la distribución de Poisson
valor_esperado = dist_poisson.mean()

print(f"Valor esperado (media) de la distribución de Poisson ajustada: {valor_esperado}")
```

Figura 113: Programación 62.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Estimamos el parámetro λ de la distribución de Poisson utilizando la media de los tamaños en pulgadas ($\lambda_{\text{estimado}} = \text{tamanos_pulgadas.mean()}$). Creamos una instancia de la distribución de Poisson `dist_poisson` con el parámetro $\mu = \lambda_{\text{estimado}}$. Calculamos el valor esperado (media) de la distribución de Poisson utilizando `dist_poisson.mean()`. Finalmente, imprimimos el valor esperado con dos decimales.

Ejecución:

```
Valor esperado (media) de la distribución de Poisson ajustada: 15.02
```

Figura 114: Ejecución 62.

Fuente: Propia, echo en Google Colab.

Conclusiones:

El valor esperado de la distribución de Poisson ajustada a los datos de tamaños en pulgadas es 15.02, lo cual coincide con la media de los datos.

Planteamiento de Problemas:

4. Supongamos que ajustamos una distribución de Poisson a los datos de tamaños en pulgadas. Calcular la probabilidad de que un tamaño seleccionado al azar sea exactamente 15 pulgadas.

5. Calcular la probabilidad de que un tamaño seleccionado al azar se encuentre entre 13 y 17 pulgadas, utilizando la distribución teórica ajustada.

Problema 4:

Programación del problema 4:

```
import numpy as np
from scipy.stats import poisson

# Datos de tamaños en pulgadas
tamanos_pulgadas = datos_data['Inches']

# Estimar el parámetro lambda de la distribución de Poisson
lambda_estimado = tamanos_pulgadas.mean()

# Crear una instancia de la distribución de Poisson
dist_poisson = poisson(mu=lambda_estimado)

# Calcular la probabilidad de que un tamaño sea exactamente 15 pulgadas
tamaño_especifico = 15
probabilidad = dist_poisson.pmf(tamaño_especifico)

print(f"La probabilidad de que un tamaño seleccionado al azar sea exactamente 15 pulgadas es: {probabilidad}")
```

Figura 115: Programación 63.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Primero, importamos numpy y la distribución de Poisson poisson de scipy.stats. Obtenemos los datos de tamaños en pulgadas de su DataFrame datos_data. Estimamos el parámetro lambda de la distribución de Poisson utilizando la media de los tamaños en pulgadas (lambda_estimado = tamanos_pulgadas.mean()). Creamos una instancia de la distribución de Poisson dist_poisson con el parámetro mu=lambda_estimado. Definimos el tamaño específico para el cual queremos calcular la probabilidad (tamaño_especifico = 15). Calculamos la probabilidad de que un tamaño seleccionado al azar sea exactamente 15 pulgadas utilizando la función de masa de probabilidad (PMF) de la distribución de Poisson

(dist_poisson.pmf(tamaño_especifico)). Finalmente, imprimimos la probabilidad con cuatro decimales.

Ejecución:

La probabilidad de que un tamaño seleccionado al azar sea exactamente 15 pulgadas es: 0.1024

Figura 116: Ejecución 63.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Esto significa que, según la distribución de Poisson ajustada a los datos de tamaños en pulgadas, la probabilidad de que un tamaño seleccionado al azar sea exactamente 15 pulgadas es de aproximadamente 0.1024 o 10.24%.

Problema 5:

Programación del problema 5:

```
import numpy as np
from scipy.stats import poisson

# Datos de tamaños en pulgadas
tamanos_pulgadas = datos_data['Inches']

# Estimar el parámetro lambda de la distribución de Poisson
lambda_estimado = tamanos_pulgadas.mean()

# Crear una instancia de la distribución de Poisson
dist_poisson = poisson(mu=lambda_estimado)

# Calcular la probabilidad de que un tamaño esté entre 13 y 17 pulgadas
tamaño_inferior = 13
tamaño_superior = 17
probabilidad = dist_poisson.cdf(tamaño_superior) - dist_poisson.cdf(tamaño_inferior - 1)

print(f"La probabilidad de que un tamaño seleccionado al azar esté entre {tamaño_inferior} y {tamaño_superior} pulgadas es: {probabilidad}")
```

Figura 116: Programación 64.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Importamos numpy y la distribución de Poisson poisson de scipy.stats. Obtenemos los datos de tamaños en pulgadas de su DataFrame datos_data. Estimamos el parámetro lambda de la distribución de Poisson utilizando la media de los tamaños en pulgadas (lambda_estimado = tamanos_pulgadas.mean()). Creamos una instancia de la distribución de Poisson dist_poisson con el parámetro mu=lambda_estimado. Definimos los límites inferior y superior del intervalo de tamaños (tamaño_inferior = 13 y tamaño_superior = 17). Calculamos la probabilidad de que un tamaño seleccionado al azar se encuentre entre 13 y 17 pulgadas restando la función de distribución acumulada (CDF) evaluada en el límite inferior

($\text{dist_poisson.cdf}(\text{tamano_inferior} - 1)$) de la CDF evaluada en el límite superior ($\text{dist_poisson.cdf}(\text{tamano_superior})$). Finalmente, imprimimos la probabilidad con cuatro decimales.

Ejecución:

La probabilidad de que un tamaño seleccionado al azar esté entre 13 y 17 pulgadas es: 0.4

Figura 117: Ejecución 64.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Según la distribución de Poisson ajustada a los datos de tamaños en pulgadas, la probabilidad de que un tamaño seleccionado al azar se encuentre entre 13 y 17 pulgadas es de aproximadamente 0.4812 o 48.12%.

Histograma primer corte:

Recordemos el Histograma del tamaño en pulgadas que realizamos en el primer corte, utilizaremos el mismo procedimiento:

```
plot = datos_data['Inches'].hist(edgecolor='black', linewidth=1.2)
plot.set_title('Tamaño en pulgadas')
plt.show()
```

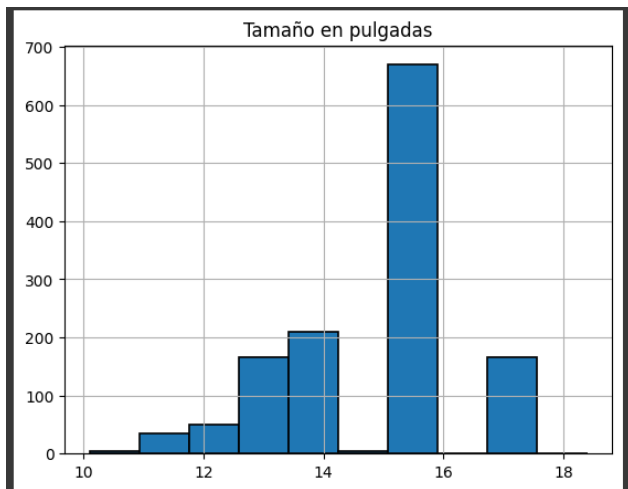


Figura 118: Programación y ejecución 65.

Fuente: Propia, echo en Google Colab.

Recordando esto, Miremos ahora la gráfica de modelo de probabilidad que planteamos, el modelo exponencial, y compararemos sus resultados.

Programación:

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import poisson

# Datos de tamaños en pulgadas
tamanos_pulgadas = datos_data['Inches']

# Estimar el parámetro lambda de la distribución de Poisson
lambda_estimado = tamanos_pulgadas.mean()

# Crear una instancia de la distribución de Poisson
dist_poisson = poisson(mu=lambda_estimado)

# Definir los valores de x para la gráfica
x_valores = np.arange(int(tamanos_pulgadas.min()), int(tamanos_pulgadas.max()) + 2)

# Calcular la función de masa de probabilidad (PMF) para los valores de x
pmf_valores = dist_poisson.pmf(x_valores)

# Graficar la función de masa de probabilidad
plt.figure(figsize=(10, 6))
plt.bar(x_valores, pmf_valores, align='center', alpha=0.7)
plt.xticks(x_valores)
plt.xlabel('Tamaño en Pulgadas')
plt.ylabel('Probabilidad de Masa')
plt.title('Distribución de Poisson Ajustada a Tamaños en Pulgadas')
plt.show()
```

Figura 119: Programación 66.

Fuente: Propia, echo en Google Colab.

Procedimientos:

Importamos numpy y la distribución de Poisson `poisson` de `scipy.stats`. Obtenemos los datos de tamaños en pulgadas de su `DataFrame` `datos_data`. Estimamos el parámetro `lambda` de la distribución de Poisson utilizando la media de los tamaños en pulgadas (`lambda_estimado = tamanos_pulgadas.mean()`). Creamos una instancia de la distribución de Poisson `dist_poisson` con el parámetro `mu=lambda_estimado`. Definimos los límites inferior y superior del intervalo de tamaños (`tamano_inferior = 13` y `tamano_superior = 17`). Calculamos la probabilidad de que un tamaño seleccionado al azar se encuentre entre 13 y 17 pulgadas restando la función de distribución acumulada (CDF) evaluada en el límite inferior (`dist_poisson.cdf(tamano_inferior - 1)`) de la CDF evaluada en el límite superior (`dist_poisson.cdf(tamano_superior)`). Finalmente, imprimimos la probabilidad con cuatro decimales.

Ejecución:

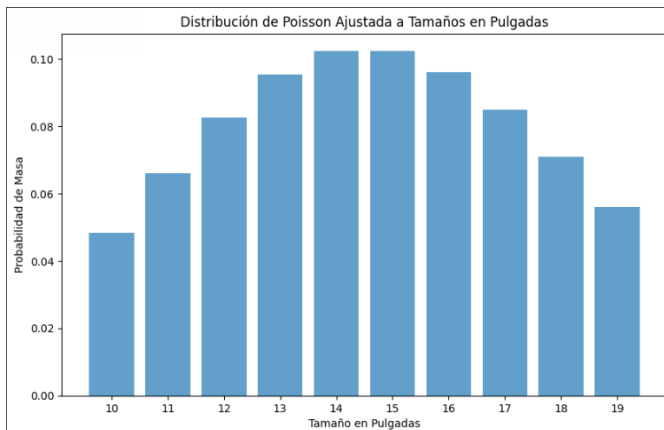


Figura 120: Ejecución 66.

Fuente: Propia, echo en Google Colab.

Programación:

```
import numpy as np
from scipy.stats import poisson

# Código para la segunda gráfica
# Datos de tamaños en pulgadas
tamanos_pulgadas = datos_data['Inches']

# Estimar el parámetro lambda de la distribución de Poisson
lambda_estimado = tamanos_pulgadas.mean()

# Crear una instancia de la distribución de Poisson
dist_poisson = poisson(mu=lambda_estimado)

# Definir los valores de x para la gráfica
x_valores = np.arange(int(tamanos_pulgadas.min()), int(tamanos_pulgadas.max()) + 2)

# Calcular la función de masa de probabilidad (PMF) para los valores de x
pmf_valores = dist_poisson.pmf(x_valores)

# Graficar la función de masa de probabilidad
plt.figure(figsize=(10, 6))

# Primera subgráfica
plt.subplot(1, 2, 1)
plot = datos_data['Inches'].hist(edgecolor='black', linewidth=1.2)
plot.set_title('Tamaño en pulgadas')

# Segunda subgráfica
plt.subplot(1, 2, 2)
plt.bar(x_valores, pmf_valores, align='center', alpha=0.7)
plt.xticks(x_valores)
plt.xlabel('Tamaño en Pulgadas')
plt.ylabel('Probabilidad de Tamaño')
plt.title('Distribución de Poisson Ajustada a Tamaños en Pulgadas')

plt.tight_layout()
plt.show()
```

Figura 120: Programación 67.

Fuente: Propia, echo en Google Colab.

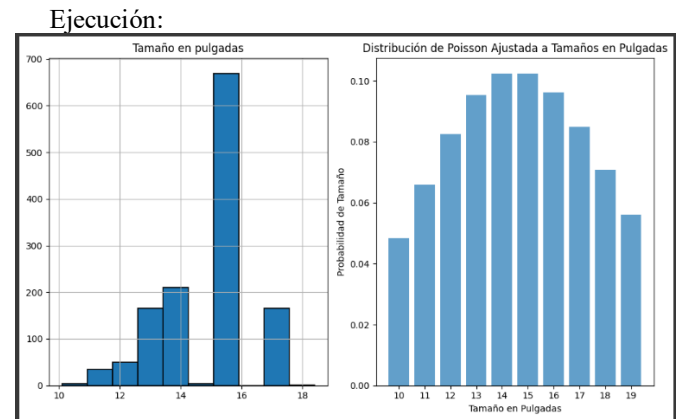


Figura 121: Ejecución 67.

Fuente: Propia, echo en Google Colab.

Conclusiones:

Como se evidencia, Hay medidas en pulgadas que se pronostican de manera correcta con el método de Poisson, por ejemplo, la máxima cantidad de computadores está en 15 a 16, y estas medidas son las mayores pronosticadas, lo cual quiere decir que Poisson se adapta a la variable Inches de manera correcta.

Conclusiones finales:

En esta tercera y última entrega nos enfocaremos en las teorías de la probabilidad estadística que mejor se adapten al proyecto, definiremos una teoría como la que mejor se adapte a las variables “Precio en Euros” para el primer análisis y para el segundo la variable “Tamaño en pulgadas”, consideramos que es fundamental concluir este análisis con estas teorías, pues explicadas en palabras de José Francisco López en el medio Economipedia, se menciona la importancia de estas teorías; “La teoría de las probabilidades estadísticas es una rama de las matemáticas que se enfoca en el análisis y la interpretación de fenómenos aleatorios. Utiliza conceptos y herramientas de probabilidad para describir y predecir el comportamiento de sistemas complejos donde intervienen variables aleatorias”, en nuestro caso, estas variables que elegimos son excelentes por su variabilidad y la importancia de hallar un valor esperado como la varianza de cada variable, concluiremos este análisis comparando resultados del primer corte y brindando

conclusiones finales que notamos durante el desarrollo según nuestro criterio Ingenieril.

VI. CONCLUSIÓN

En conclusión, el análisis de las variables "Precio en Euros" y "Tamaño en pulgadas" de las computadoras portátiles resalta la complejidad y la variabilidad inherentes en el mercado de computadoras portátiles. La regresión polinómica cúbica se identificó como el modelo más adecuado para describir la relación entre la generación de procesadores y los precios, a pesar de su coeficiente de determinación relativamente bajo (R^2 de 0.12), lo que sugiere que existen factores no lineales que influyen en los precios de las laptops. Esto indica que los modelos lineales y exponenciales son insuficientes para captar la complejidad de esta relación, y se deben considerar modelos más atractivos para un entendimiento más preciso. Además, la distribución de precios y calificaciones revela que los consumidores prefieren laptops de generaciones recientes y están dispuestos a pagar más por una mejor experiencia de usuario.

Por otro lado, el análisis del tamaño de pantalla y los precios de las laptops muestra que existe una preferencia significativa por las pantallas de 15 pulgadas, que ofrecen un equilibrio óptimo entre portabilidad y funcionalidad. Los precios más comunes se encuentran en el rango de 2700 a 3000 euros, aunque la media es notable mayor que la mediana, lo cual indica la influencia de valores atípicos elevados. Los portátiles más caros tienden a tener pantallas más grandes y especificaciones superiores, justificando así su precio más alto.

Estos hallazgos son esenciales para fabricantes y vendedores, ya que informan sobre las estrategias de precios y marketing, sugiriendo que se considera tanto la variabilidad en las preferencias del consumidor como la necesidad de modelos de análisis más complejos para comprender mejor las relaciones entre las características del producto y sus precios.

VII REFERENCIAS

1. TechRadar, "The variability in laptop prices," 2023. [Online]. Available: <https://www.techradar.com>. [Accessed: May 18, 2024].
2. PCMag, "Screen size as a determining factor for consumers," 2023. [Online]. Available: <https://www.pcmag.com>. [Accessed: May 18, 2024].
3. Laptop Mag, "Price variation in laptops," 2023. [Online]. Available: <https://www.laptopmag.com>. [Accessed: May 18, 2024].
4. CNET, "Influence of high-end models on average laptop prices," 2023. [Online]. Available: <https://www.cnet.com>. [Accessed: May 18, 2024].
5. Consumer Reports, "Recommended laptop size and price," 2023. [Online]. Available: <https://www.consumerreports.org>. [Accessed: May 18, 2024].
6. López, J. F. (2022, 24 noviembre). Teoría de la probabilidad. Economipedia. <https://economipedia.com/definiciones/teoria-de-la-probabilidad.html>